

Introduction Générale aux Systèmes d'Exploitation

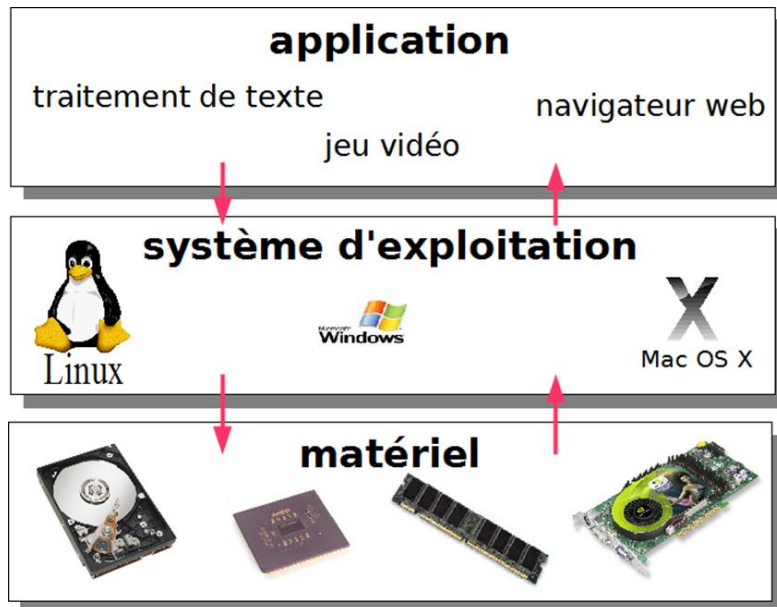
Plan

1. Définition
2. Les différents types de système d'exploitation.
3. Rôles du système d'exploitation
4. Historique
5. Structure des systèmes d'exploitation

Système d'exploitation

Définition

Un système d'exploitation (SE) en anglais (operating system) est un ensemble de programmes responsables de la liaison entre les ressources matérielles d'un ordinateur et les applications informatiques de l'utilisateur



L'utilisateur travaille avec ses logiciels.

Les logiciels communiquent avec l'OS.

L'OS échange avec l'ordinateur.

Multi utilisateurs

Un système d'exploitation qui peut supporter plusieurs sessions en même temps.

Multi tâches

Un système d'exploitation est multitâche s'il permet d'exécuter, de façon apparemment simultanée, plusieurs programmes informatiques. On parle également de la multiprogrammation.

Multi tâches « coopératif »

Le passage de l'exécution d'un processus à un autre est appelé commutation de contexte. Ces commutations sont initiées par les processus eux-mêmes

Inconvénients :

- Processus en cours bloqué = système bloqué
- Le partage des ressources (temps CPU, mémoire, accès disque, etc.) peut être inefficace.

Windows 3.x



Multi tâches « préemptif »

le processeur signale au système d'exploitation que le processus en cours d'exécution doit être mis en pause pour permettre l'exécution d'un autre processus.

- Sauvegarde de l'état
- File d'attente
- Restauration du contexte d'exécution



Multi processeurs

Système avec plusieurs processeurs en parallèle.

- vrai multitâches (Exécution de plusieurs processus en même temps).
- Puissance de calcul plus importante.
- Disponibilité du système (en cas de panne d'un processeur)

Systèmes temps réel

prévus pour traiter des informations de manière fiable dans un temps donnés :

- Applications industrielles,
- Robotique,
- Transports, ...



Systèmes embarqués

prévus pour fonctionner sur :

- des machines de petite taille (, téléphone, ...)
- des appareils électroniques autonomes (sondes spatiales, robot, ordinateur de bord de véhicule, ...)

Autonomie réduite = gestion avancée de l'énergie



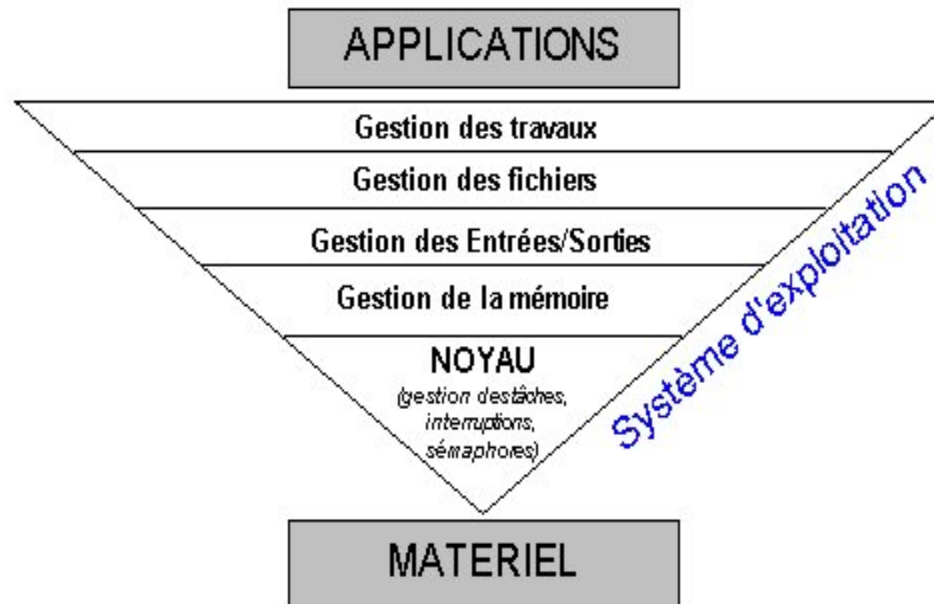
Rôles du système d'exploitation

La gestion du processeur: le système d'exploitation est chargé de gérer l'allocation du processeur entre les différents programmes grâce à un algorithme d'ordonnancement .

Gestion de la mémoire : le système d'exploitation est chargé de gérer l'espace mémoire alloué à chaque application.

Gestion des entrées/sorties: le système d'exploitation permet de contrôler l'accès des programmes aux ressources matérielle par l'intermédiaire des pilotes.

Gestion des fichiers : le système d'exploitation gère la lecture et l'écriture dan le système de fichier et des droits d'accès aux fichiers par les utilisateurs et les applications.



Historique

Première génération (1945-1955).



Deuxième génération (1955-1965)



Troisième génération (1965-1980)



La quatrième génération (après les années 1980)

Historique: La première génération (1945-1955).

- Moteurs de calcul utilisant des relais mécaniques (temps de cycles en secondes) remplacés ensuite par des tubes à vide
- Pas de système d'exploitation.
- Tout programme était conçu en langage machine (pas de langage abstrait)

Historique: Deuxième génération (1955-1965)

- La deuxième génération d'ordinateurs est basée sur l'invention du transistor en 1947. Cela permet de remplacer le fragile et encombrant tube électronique par un composant plus petit et fiable.
- Programmes écrits en FORTRAN puis codés sur des cartes perforées

- Circuits intégrés
- Début 60, des machines pour E/S (IBM 1401)
d'autres pour le calcul (IBM 7094)
- Multiprogrammation

Plusieurs programmes peuvent tourner en parallèle sur une même machine

- Mémoire partagée
- Mécanismes de protection
- Spool (Simultaneous Peripheral Operation On Line)
- Systèmes d'exploitation: MULTICS, UNIX

Historique: La quatrième génération (après les années 1980)

- Ordinateurs personnels
- Circuits intégrés à haute densité (Puces contenant des milliers de transistors sur 1mm² de silicium).
- Micro-ordinateurs, très peu onéreux comparés aux mini-ordinateurs de type PDP-11
- Systèmes d'Exploitation CP/M, MS-DOS, MAC OS X, Windows, UNIX, Linux...
- Avec l'évolution des réseaux de télécommunication de nouvelles catégories de SE ont apparues telles que les SE réseaux et les Systèmes Répartis ou distribués.

Les Etapes de la compilation

gcc effectue quatre étapes pour produire l'exécutable

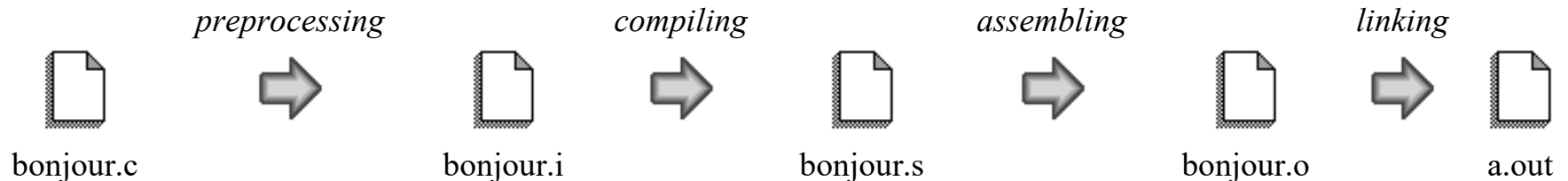
1. Passage au pré-processeur (*preprocessing*)
2. Compilation en langage assembleur (*compiling*)
3. Conversion du langage assembleur en code machine (*assembling*)
4. Edition des liens (*linking*)

bonjour.c

```
#include <stdio.h>

int main(int argc, const char * argv[])
{
    // Afficher le mot bonjour et
    sauter une ligne
    printf("Bonjour!!!\n");
    return 0;
}
```

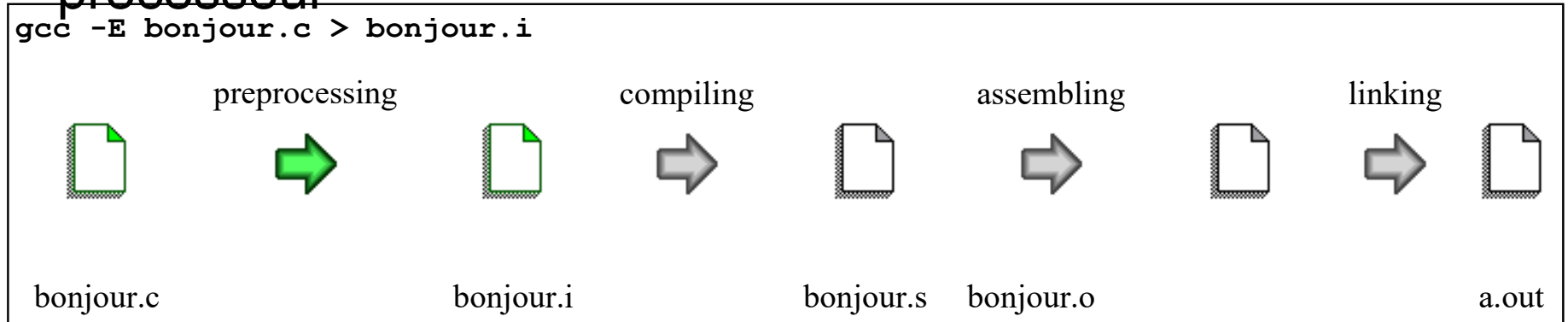
gcc bonjour.c



Compilation

Etape 1: Passage au pré- processeur

```
gcc -E bonjour.c > bonjour.i
```



```
gcc -E bonjour.c >  
bonjour.i
```

```
#include <stdio.h>  
#1 "bonjour.c"  
int main(int argc, const char * argv[])  
{  
    printf("Bonjour!!!\n");  
    return 0;  
}
```

Etape 2: Compilation en langage assembleur

gcc -S bonjour.i

preprocessing

compiling

assembling

linking



bonjour.c

bonjour.i

bonjour.

bonjour.

a.out

gcc -S bonjour.i

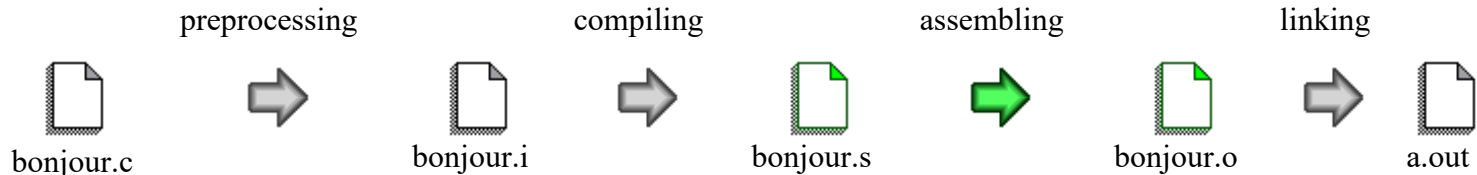
bonjour.s

```
bonjour.s
.file "bonjour.c"
gcc2_compiled.:
.section ".rodata"
.align 8
.LLC0:
.asciz "bonjour\n"
.section ".text"
.align 4
.global main
.type
main:
function
!#PROLOGUE# 0 04
!#PROLOGUE# 1
call printf, 0
nop
mov 0, %i0
b .LL2
.LL2:
nop
ret
```

Compilation

Etape 3: Transformation de l'assembleur en code machine

`gcc -c bonjour.s`



`gcc -c bonjour.s`

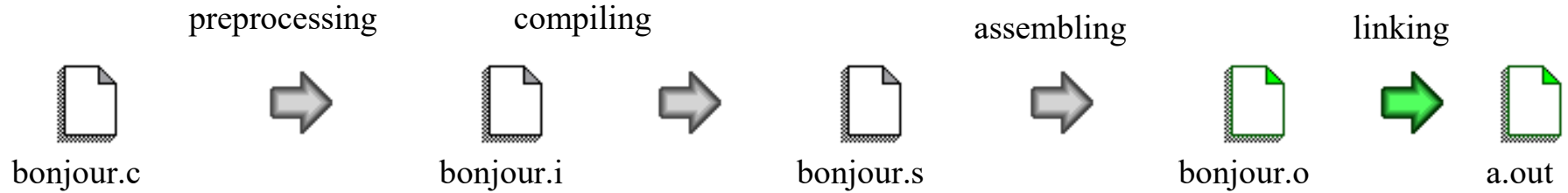
```

od -x bonjour.o
000000 7f45 4c46 0102 0100 0000 0000 0000 0000
000020 0001 0002 0000 0001 0000 0000 0000 0000
000040 0000 01ac 0000 0000 0034 0000 0000 0028
000060 0008 0001 002e 7368 7374 7274 6162 002e
000100 7465 7874 002e 726f 6461 7461 002e 7379
000120 6d74 6162 002e 7374 7274 6162 002e 7265
000140 6c61 2e74 6578 7400 2e63 6f6d 6d65 6e74
000160 0000 0000 9de3 bf90 1300 0000 9012 6000
000200 4000 0000 0100 0000 b010 2000 1080 0002
000220 0100 0000 81c7 e008 81e8 0000 0000 0000
000240 626f 6e6a 6f75 720a 0000 0000 0000 0000
000260 0000 0000 0000 0000 0000 0000 0000 0001
000300 0000 0000 0000 0000 0400 fff1 0000 000b
000320 0000 0000 0000 0000 0000 0002 0000 0000
000340 0000 0000 0000 0000 0300 0002 0000 0000
  
```

Compilation

Etape 4 : Edition des liens

gcc bonjour.o



Compilation

Etape 1: pré-processeur

- **gcc -E bonjour.c -o bonjour.i**
- **bonjour.i**
- **Cat bonjour.i**

Etape 2: Compilation en langage assembleur

- **gcc -S bonjour.i -o bonjour.s**
- **bonjour.s**

Etape 3: Transformation de l'assembleur en code machine

- **gcc -c bonjour.s**
- **bonjour.o** **od -x bonjour.o**

Etape 4 : Edition des liens

- **gcc bonjour.o**
- **a.out**

Chapitre 2:La gestion des processus

SYSTÈMES D'EXPLOITATION AVANCÉES

Plan

1. Définition
 - a. Processus
2. Le modèle des processus
 - a. L'arborescence
 - b. Le PCB
 - c. Les états d'un processus
 - d. Commutation de contexte
3. Ordonnancement des processus

Définition : processus

Un processus (Process en anglais) est un programme en cours d'exécution. Tout processus possède des caractéristiques propres (Ex. un numéro d'identification), des ressources qu'il utilise (comme des fichiers ouverts) et se trouve à tout moment dans un état (en exécution ou en attente ...).

Un processus est constitué d' :

- Un code exécutable du programme en exécution.
- Un contexte qui est une image décrivant l'environnement du processus.

Définition : processus

- Caractéristiques statiques, c'est-à-dire ne variant pas au cours de sa vie, sont:
 - Un numéro unique: PID (Process IDentifier),
 - Un propriétaire déterminant les droits d'accès du processus aux ressources : ouverture de fichiers...
 - Un processus parent dont il hérite la plupart des caractéristiques,
 - Un terminal d'attache pour les entrées/sorties.
- Caractéristiques dynamiques sont:
 - Priorité, environnement d'exécution...
 - Quantité de ressources consommées (temps unité centrale utilisé...)

Image mémoire d'un processus

L'espace mémoire alloué à un processus, dit image mémoire (Memory Map) du processus, il est divisé en un ensemble de parties :

1. Segment de code (Text section) : Copie du segment de code du fichier exécutable

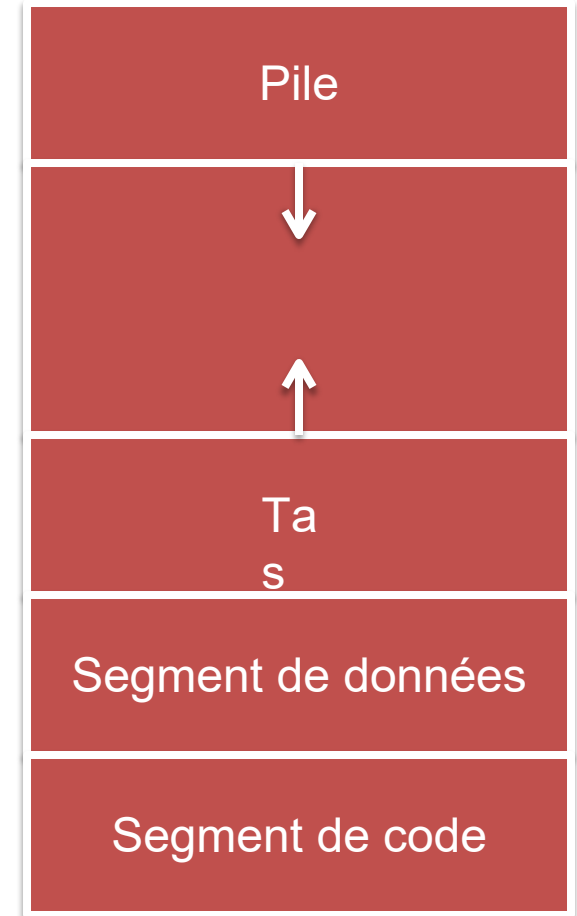
2. Segment de données (Data section) : qui contient l'ensemble des constantes et variables déclarées.

3. Pile (Stack) : qui permet de stocker :

- les valeurs des registres.
- les variables locales et paramètres de fonctions.
- les adresses de retour des fonctions.

4. Tas (Heap) :

zone de la mémoire allouée dynamiquement.



Bloc de contrôle de processus

Chaque processus est représenté dans le système d'exploitation par une structure de données contenant toute les informations décrivant le contexte d'un processus appelé bloc de contrôle (Process Control Bloc: PCB).

PCB: contient plusieurs informations concernant un processus spécifique :

- **L'état du processus:** nouveau, prêt, en exécution, en attente, arrêté...
- **PID, UID, GID, PPID**
- **Compteur d'instructions:** : indique l'adresse de la prochaine instruction à exécuter par ce processus.
- **Informations sur l'ordonnancement de la CPU:** information concernant la priorité du processus.
- **Informations sur la gestion de la mémoire**
- **Informations sur l'état des E/S:** liste des périphériques E/S allouées à ce processus, une liste des fichiers ouverts, etc.

PCB	
Pointeur	État du processus
Numéro du processus	
Compteur d'instructions	
Registres	
Limite de la mémoire	
Liste des fichiers ouverts	
...	

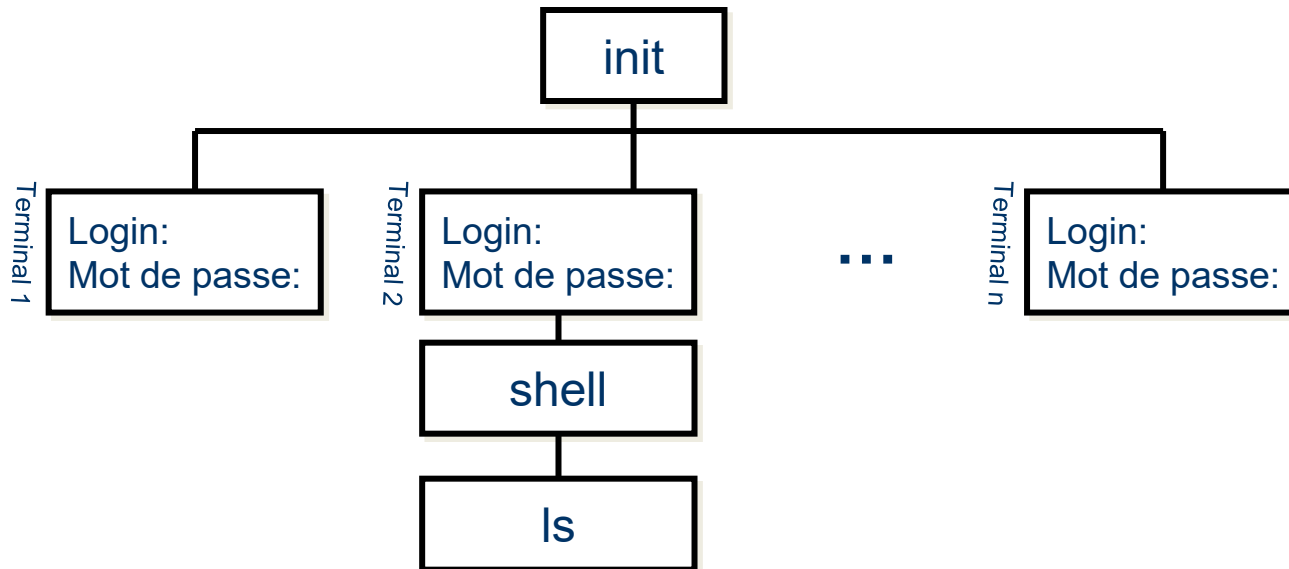
Contexte d'un Processus

Le contexte d'un processus est l'ensemble des données qui permettent de reprendre l'exécution d'un processus qui a été interrompu. Il est formé de :

- Mot d'état (Program Status Word PSW),
- registres généraux.
- compteur ordinal (Program Counter PC). Contient l'adresse mémoire de la prochaine instruction à exécuter.
- registre d'instruction (Instruction Register IR). Contient l'instruction en cours de traitement.
- registre d'état (FLAGS register). Ensemble de bits représentant des drapeaux.
- pointeurs de pile (Stack Pointer SP). Contient l'adresse du sommet de la pile.
- mode d'exécution (MODE). En mode esclave, les instructions privilégiées sont interdites, et en mode maître les restrictions disparaissent.
- masque d'interruptions (Interrupt Mask IM). Indique le niveau de priorité du processus courant, qui est utilisée pour savoir si une interruption doit être prise en compte immédiatement, ou alors être retardée (masque).

Arborescence des processus

- Les processus sont organisés sous forme d'une arborescence ou chaque processus a un seul père et peut avoir plusieurs fils.
- Un processus est identifié par un PID (Process IDentifier) et un PPID (Parent Process IDentifier).
- Exemple: L'arborescence des processus sous Linux



Arborescence des processus unix

```
[root@localhost ~]# pstree
init--VGAAuthService
      |--acpid
      |--anacron
      |--atd
      |--auditd--audispd--{audispd}
              |--{auditd}
      |--automount--4*[{automount}]
      |--avahi-daemon--avahi-daemon
      |--bonobo-activati--{bonobo-activati}
      |--bt-applet
      |--clock-applet
      |--crond
      |--cupsd
      |--2*[dbus-daemon]
      |--dbus-launch
      |--eggcup
      |--escd--{escd}
      |--events/0
      |--gam_server
      |--gconfd-2
      |--gdm-binary--gdm-binary--Xorg
              |--gnome-session--ssh-agent
```

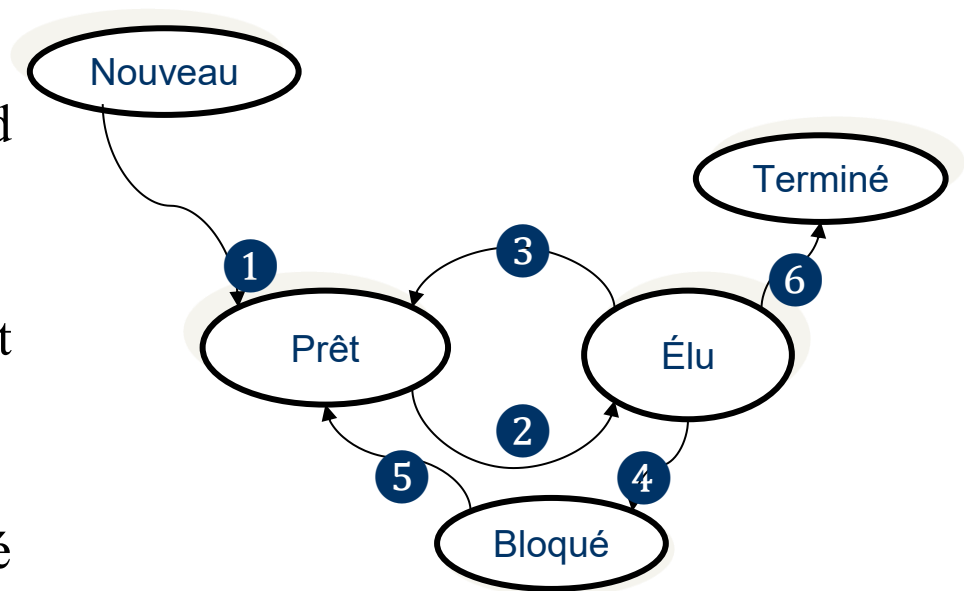
états d'un processus

Lorsqu'un processus s'exécute; il change d'état. Il peut se trouver dans l'un des trois états principaux suivants:

Prêt (Ready) : le processus attend son tour pour s'exécuter

Élu (Running) : les instructions sont en cours d'exécution.

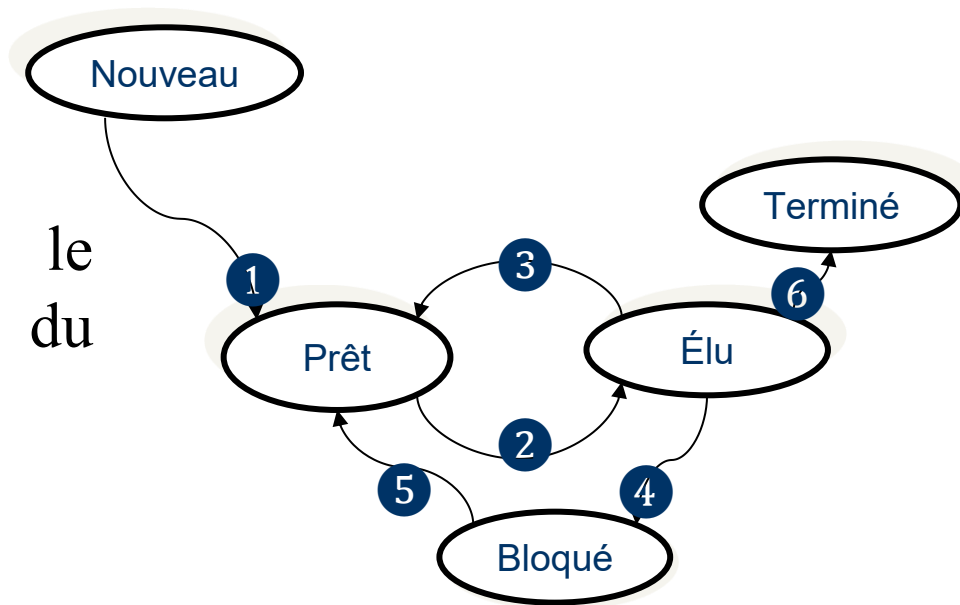
Bloqué (Sleep) : le processus bloqué en attente d'événement: signal, E/S, ...



Graphe des états d'un processus

Transitions

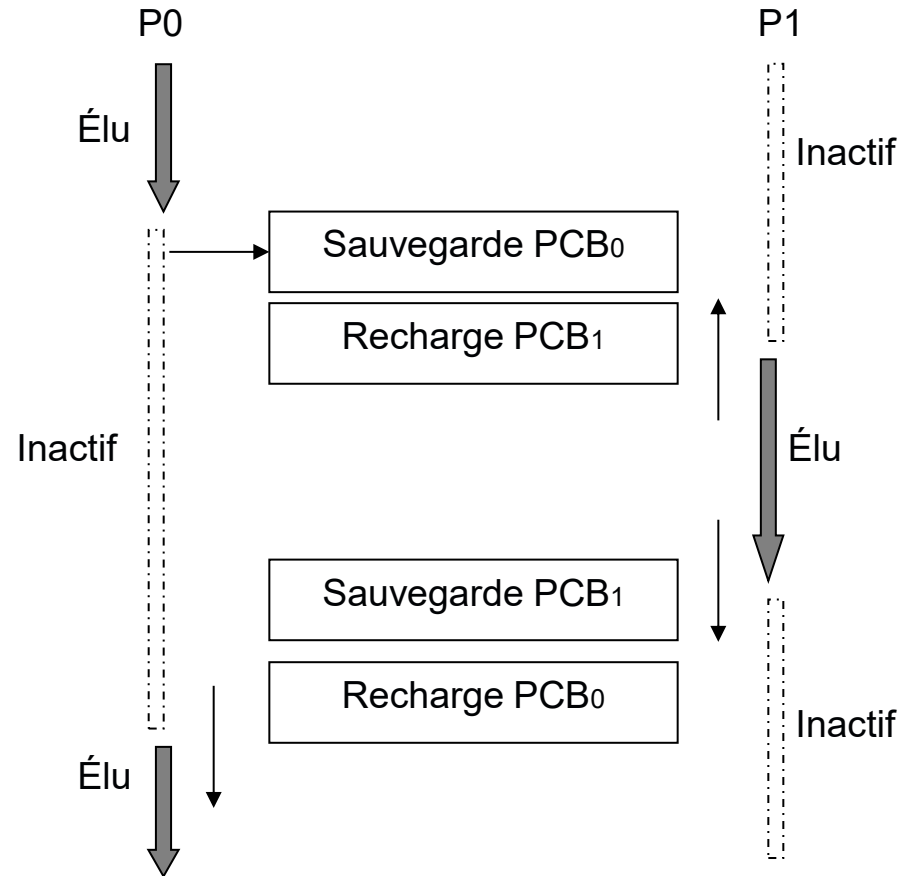
1. Création du processus
2. Allocation du processeur
3. Fin du temps alloué sur le processeur, l'exécution processus n'est pas terminée
4. Opération E/S
5. Fin opération E/S
6. Exécution terminée



Graphe des états d'un processus

Commutation de contexte

- Sur un système multiprogrammé, le SE doit redonner le contrôle du processeur d'un processus à un autre en effectuant des commutations de contexte.
- La commutation de contexte consiste à mémoriser le PCB du processus courant et charger le PCB du processus à élire.



Commutation de contexte

Le basculement d'un processus à l'autre est géré par le noyau.

- Suspendre le processus P0
- Mettre à jour le PCB de P0
- Restaurer le PCB de P1
- Reconfigurer l'espace mémoire
- Démarrer P1

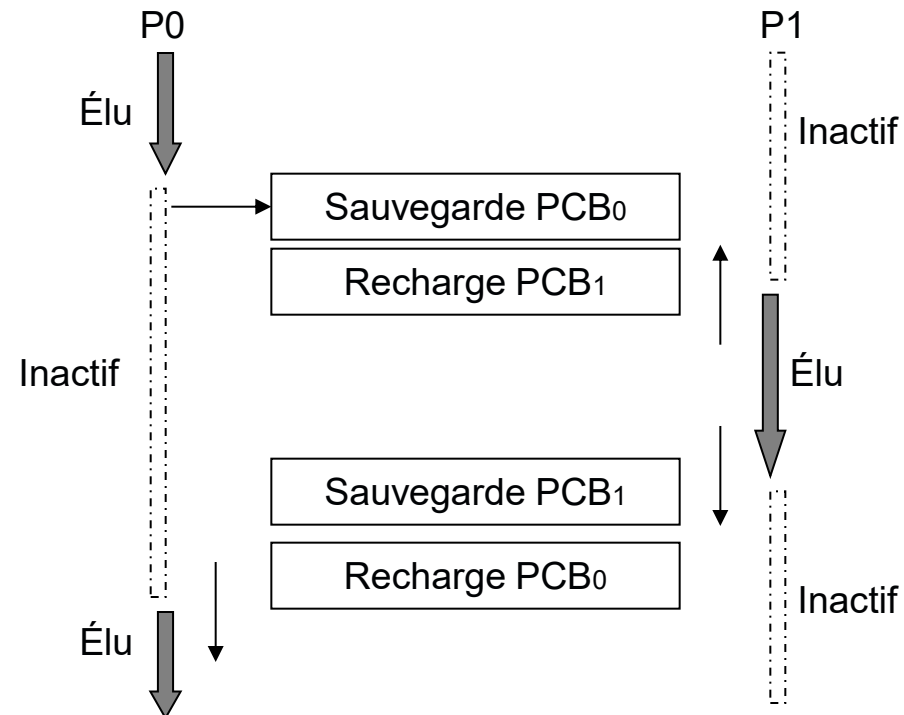


table de processus :

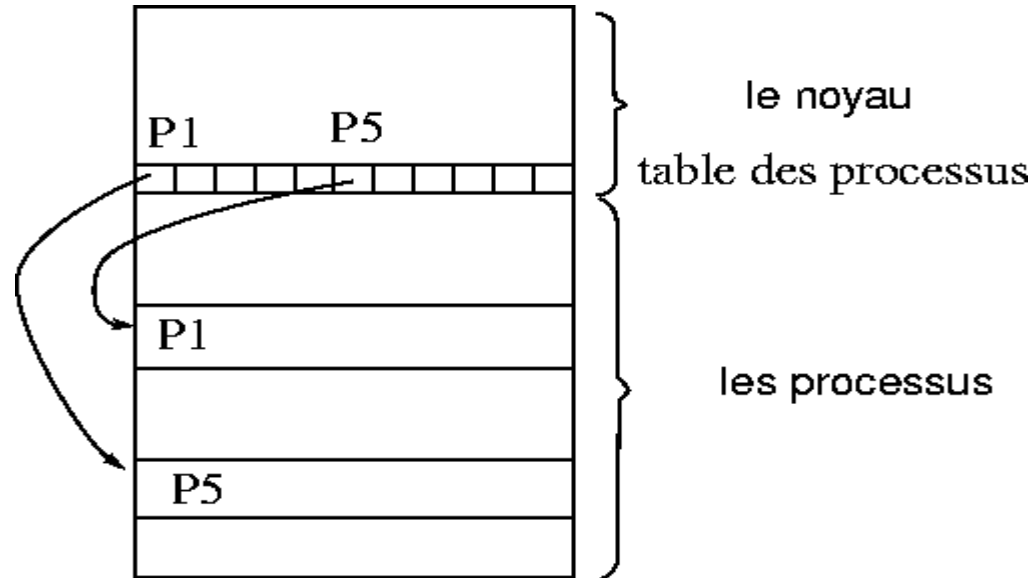
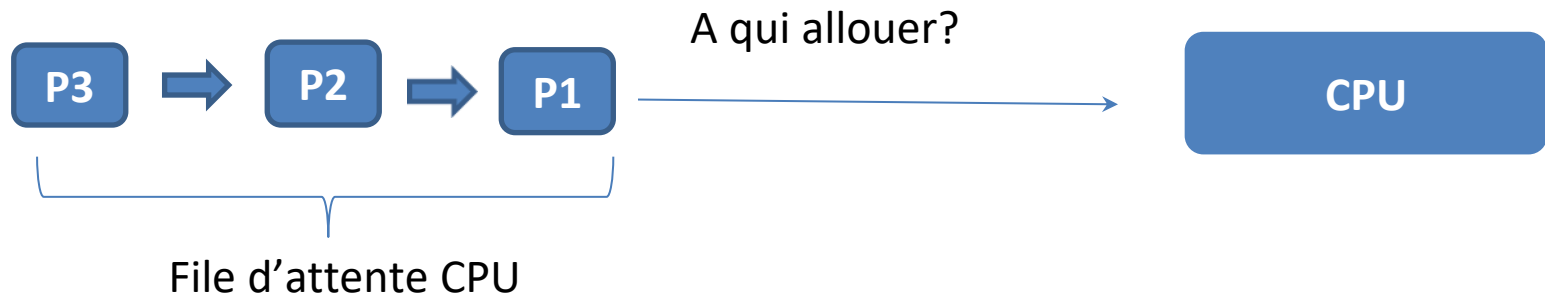


Tableau processus					
Processus	Etat	Compteur ordinal	Allocation mémoire	État des fichiers	Autres...
P1	Prêt	F8 B22 C	F800	Clients : ouvert	...
P2	Bloqué	A5 F4 6	E458	-	...
...	...				

Ordonnancement

Lorsqu'un ordinateur est multiprogrammé, il possède fréquemment plusieurs processus en concurrence pour l'obtention de temps processeur.



→ S'il n'y a qu'un seul processeur, un choix de processus à exécuter doit être fait .

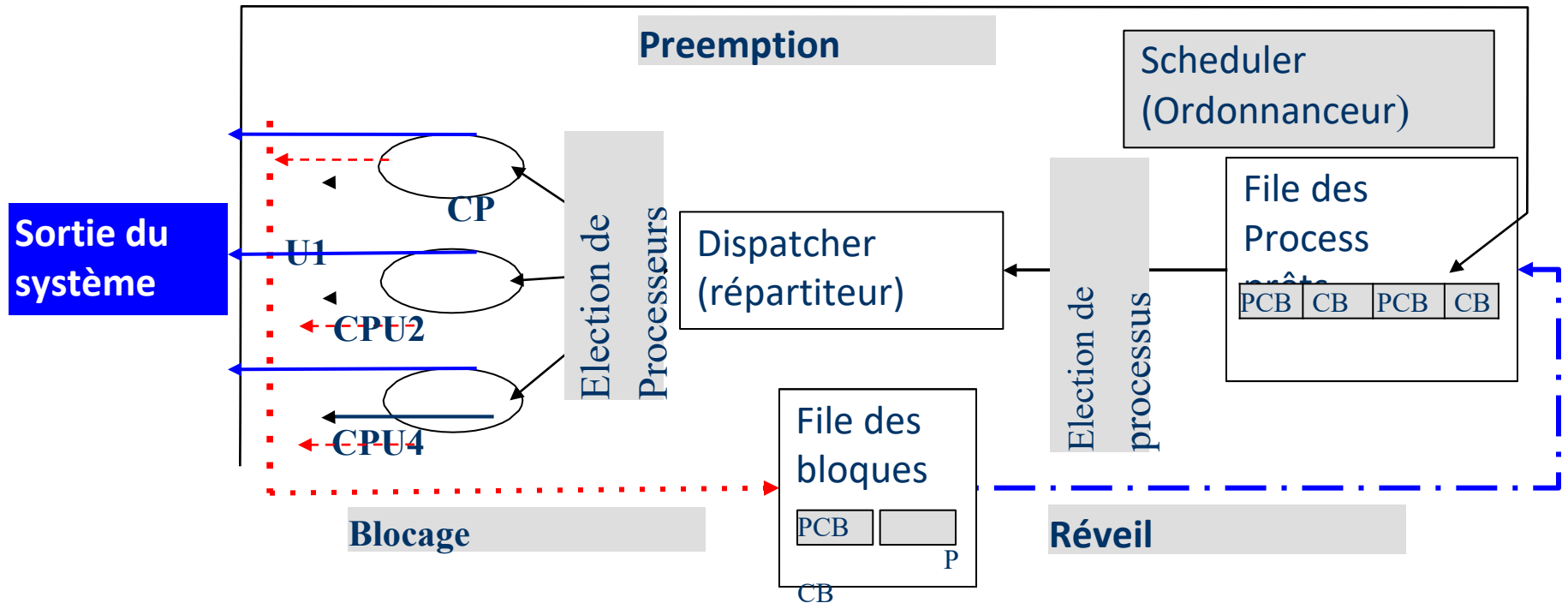
La partie du système d'exploitation qui effectue ce choix se nomme l'ordonnanceur (scheduler) et l'algorithme qu'il emploie s'appelle algorithme d'ordonnancement .

Ordonnancement des processus

Les algorithmes d'ordonnancement:

- **FCFS:** First Come First Served
- **SJF:** Shortest Job First
- **RR:** Round-Robin
- **Ordonnancement par priorité**

Les entités responsables de l'ordonnancement



Critères d'Ordonnancement

- Utilisation du CPU
- Débit (Throughput): le nombre moyen de processus traités par unité de temps.
- Temps de rotation (Turnaround time)
- Temps d'attente
- Temps de réponse

Critères d'optimisation: max/min

- Les critères d'ordonnancement peuvent être ramenés à des problèmes d'optimisation. Chaque critère devra être maximiser ou minimiser.
- Utilisation du CPU
 - Maximiser
- Débit (Throughput)
 - Maximiser
- Temps de rotation (Turnaround time)
 - Minimiser
- Temps d'attente
 - Minimiser
- Temps de réponse
 - Minimiser

Peuvent-ils tous être optimisés
en
même temps??

Avec/Sans réquisition

- **Ordonnancement sans réquisition (non préemptif):**
 - Une fois que le CPU a été allouée à un processus, ce dernier le garde jusqu'à ce qu'il le libère, soit parce qu'il a terminé, soit parce qu'il commute à l'état en attente.
 - Algorithmes simples et faciles à mettre en œuvre mais pas adaptés pour le temps partagé et seulement pour les système de traitement par lots.
- **Ordonnancement avec réquisition (préemptif):**
 - Un processus peut être suspendu à n'importe quel instant, sans avoir été prévenu, pour laisser la place à un autre processus.
 - Implémentation plus coûteuse à cause de la commutation de processus, utilisée dans les systèmes à temps partagé et systèmes temps-réels.
 - Nécessité d'un mécanisme de synchronisation.

1 Cas : FCFS (Premier arrivé, premier servi)

l'unité centrale est allouée selon l'ordre de soumission des processus.

On suppose que les processus arrivent en même temps à l'instant 0 et admis dans l'ordre : P1 , P3 , P2 , P4

On considère l'ensemble des processus suivants :

Processus	Temps d'exécution
P1	3
P2	9
P3	5
P4	7

Temps d'attente

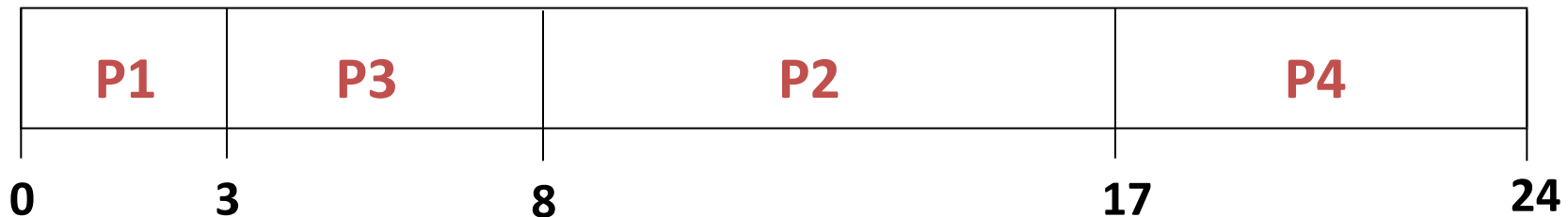
P1 = 0

P2 = 8

P3 = 3

P4 = 17

Le diagramme de Gantt pour l'ordonnancement FCFS de cet ensemble est :



Temps d'attente moyen = $(0 + 8 + 3 + 17) / 4 = 7$

2ème Cas : FCFS (Premier arrivé, premier servi)

On considère l'ensemble de processus suivant :

Processus	Temps d'exécution	Temps d'arrivée
P1	20	0
P2	12	3
P3	4	2
P4	9	5

Temps d'attente : début d'exécution – temps d'arrivée

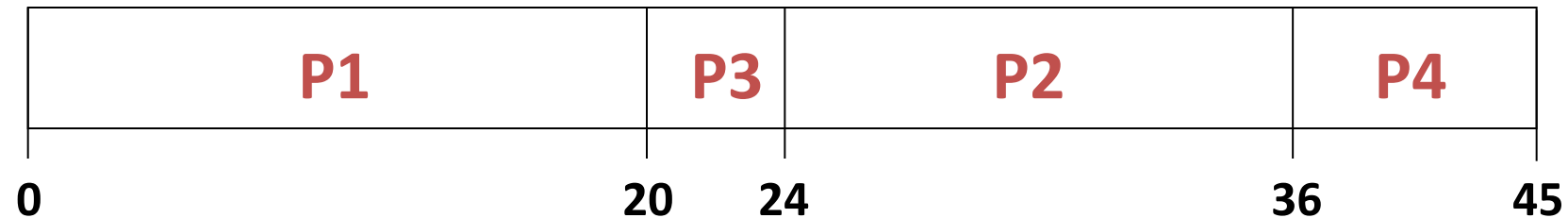
$$P1 = 0 - 0 = 0$$

$$P2 = 24 - 3 = 21$$

$$P3 = 20 - 2 = 18$$

$$P4 = 36 - 5 = 31$$

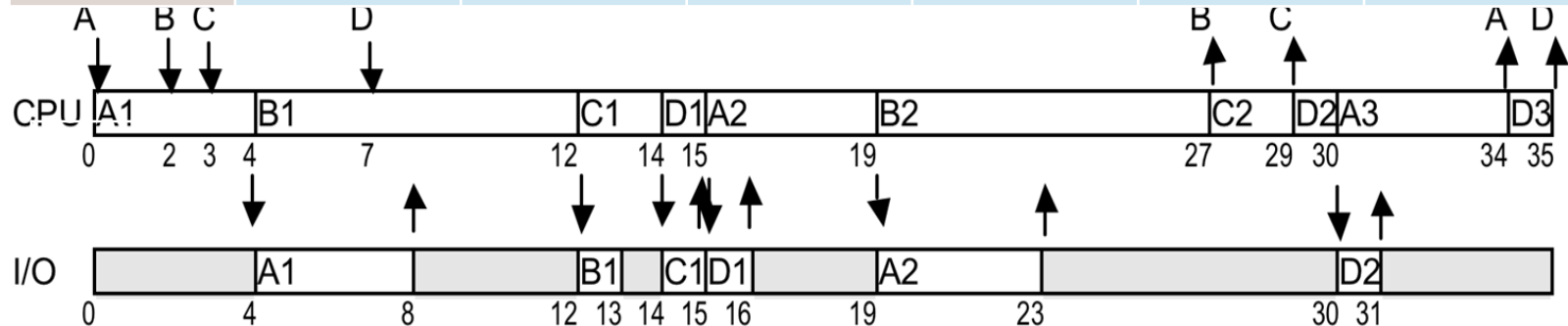
Le diagramme de Gantt pour l'ordonnancement FCFS est :



$$\text{Temps d'attente moyen} = (0 + 21 + 18 + 31) / 4 = 70 / 4 = 17,5$$

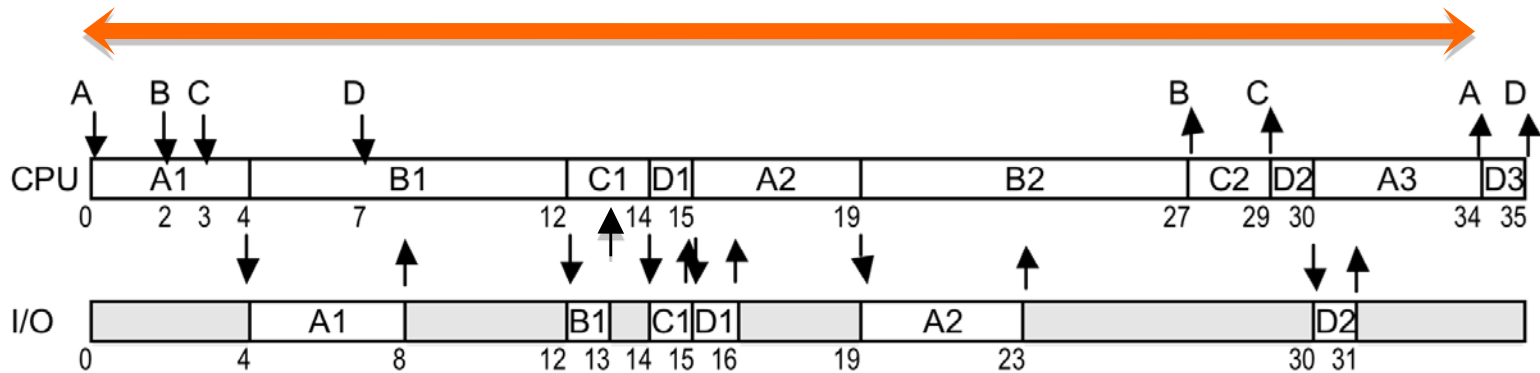
3ème Cas : FCFS (Premier arrivé, premier servi)

Process	Arrival time	1 st exec	1 st I/O	2 nd exec	2 nd I/O	3 rd exec
A	0	4	4	4	4	4
B	2	8	1	8	-	-
C	3	2	1	2	-	-
D	7	1	1	1	1	1



T=0	T=2	T=3	T=4	T=7	T=8	T=1	T=1	T=	T=1	T=1	T=1	T=1	T=2	T=2	T=3
						2	3	14	5	6	9	3	9		0
A	B	B	B	C	C	C	D	D	A	B	B	C	D	A	
		c	c	D	D	D	A	A	B	C	C	D	A		
					A	A	B	B	C	D	D	A			

3ème Cas : FCFS (Premier arrivé, premier servi)



- Turn around time (temps de traitement):

$$tat_A = 34 - 0 = 34$$

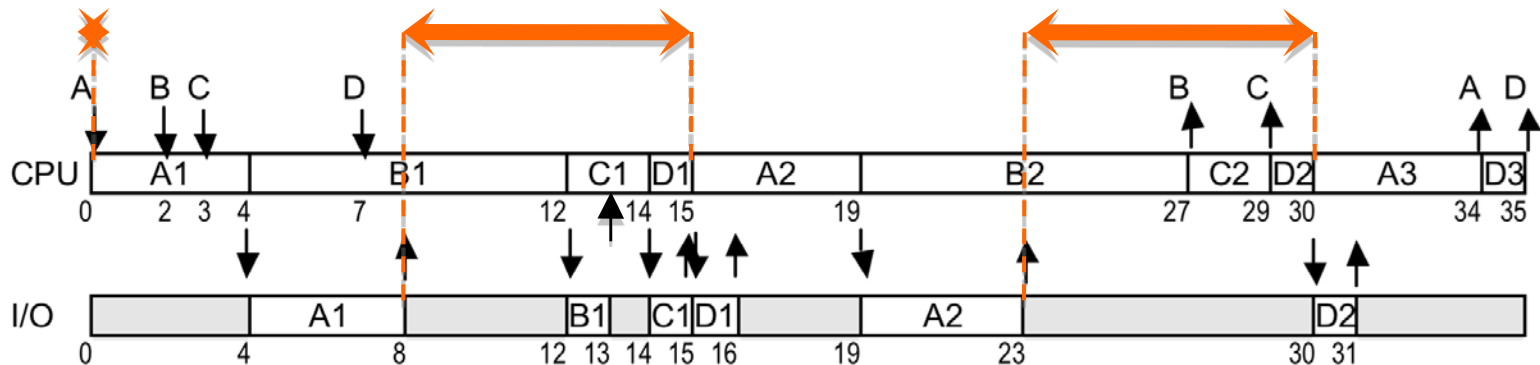
$$tat_B = 27 - 2 = 25$$

$$tat_C = 29 - 3 = 26$$

$$tat_D = 35 - 7 = 28$$

$$tat_{AVG} = (34 + 25 + 26 + 28) / 4 = 28.25$$

3ème Cas : FCFS (Premier arrivé, premier servi)



- Waiting time (temps d'attente):

$$wt_A = (0 - 0) + (15 - 8) + (30 - 23) = 14$$

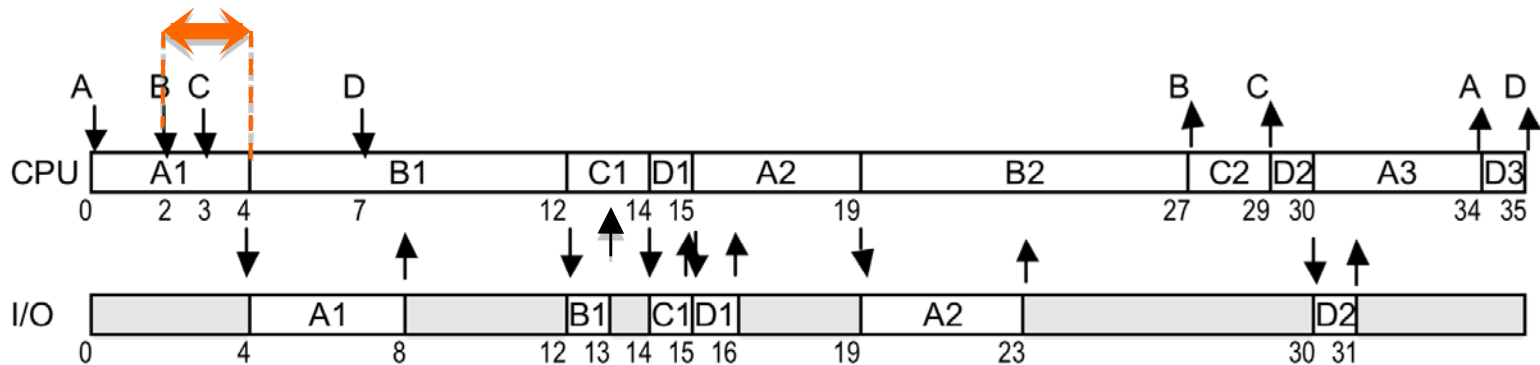
$$wt_B = (4 - 2) + (19 - 13) = 8$$

$$wt_C = (12 - 3) + (27 - 15) = 21$$

$$wt_D = (14 - 7) + (29 - 16) + (34 - 31) = 23$$

$$wt_{AVG} = (14 + 12 + 21 + 23) / 4 = 16.5$$

3ème Cas : FCFS (Premier arrivé, premier servi)



- Response time:

$$rt_A = 0 - 0 = 0$$

$$rt_B = 4 - 2 = 2$$

$$rt_C = 12 - 3 = 9$$

$$rt_D = 14 - 7 = 7$$

$$rt_{AVG} = (0 + 2 + 9 + 7) / 4 = 4.5$$

Inconvénients FIFO

- Dans cette politique, des processus de faible temps d'exécution peuvent être pénalisés parce qu'un processus de longue durée les précède dans la file .
- Le temps d'attente n'est pas proportionnel au temps d'utilisation
 - ⇒ pas équitable,
 - ⇒ temps moyen de traitement élevé

SJF (Travail le plus court d'abord) non-Préemptif

Le plus court d'abord: On exécute le processus le plus court d'abord.

On considère l'ensemble de processus suivant :

Processus	Temps d'exécution	Temps d'arrivée
P2	12	0
P3	8	3
P4	4	5
P1	10	10
P5	6	12

Temps d'attente : début d'exécution – temps d'arrivée

$P1 = 30 - 10 = 20$

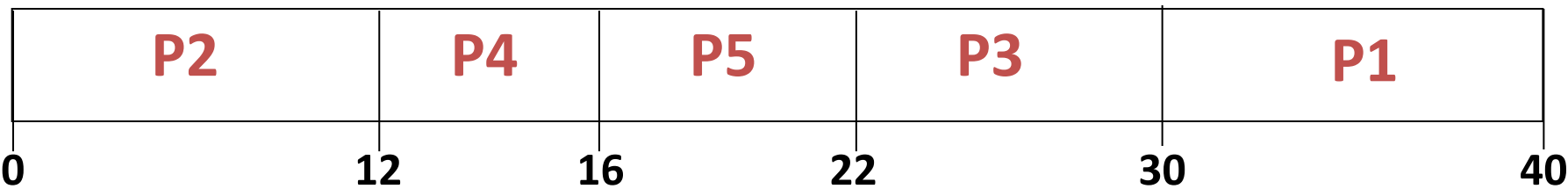
$P2 = 0 - 0 = 0$

$P3 = 22 - 3 = 19$

$P4 = 12 - 5 = 7$

$P5 = 16 - 12 = 4$

Le diagramme de Gantt avec SJF est :



Temps d'attente moyen = $(20 + 0 + 19 + 7 + 4) / 5 = 50 / 5 = 10$

SRTF (temps restant le plus court d'abord) **préemptif**

On considère l'ensemble de processus suivant

Processus	Temps d'exécution	Temps d'arrivé
US	9	0
P2	12	0
P3	8	3
P4	4	5
P1	10	10
P5	6	12

Temps d'attente : début d'exécution – temps d'arrivée

$P1 = 30 - 10 = 20$

$P2 = (0 - 0) + (21 - 3) = 18$

$P3 = (3 - 3) + (9 - 5) = 4$

$P4 = (5 - 5) = 0$

$P5 = 15 - 12 = 3$

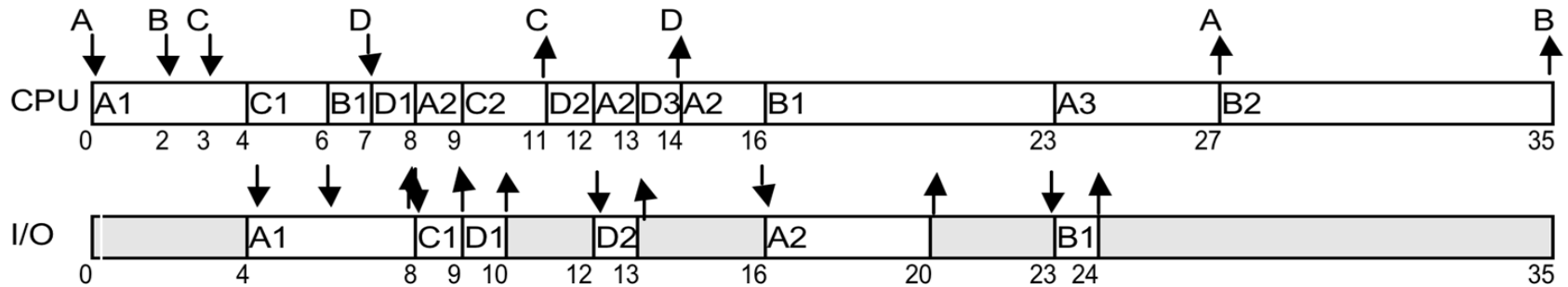
Le diagramme de Gantt avec SJF préemptif est comme suit :



Temps moyen d'attente = $(20 + 18 + 4 + 0 + 3) / 5 = 45 / 5 = 9$

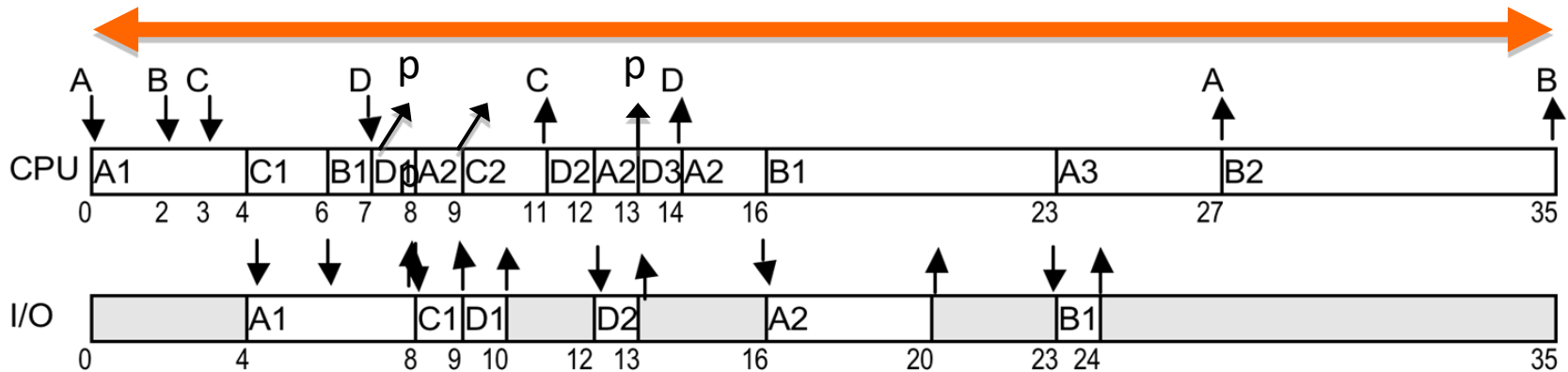
SRTF (temps restant le plus court d'abord)

Process	Arrival time	1 st exc	1 st I/O	2 nd exc	2 nd I/O	3 rd exc
A	0	4	4	4	4	4
B	2	8	1	8	-	-
C	3	2	1	2	-	-
D	7	1	1	1	1	1



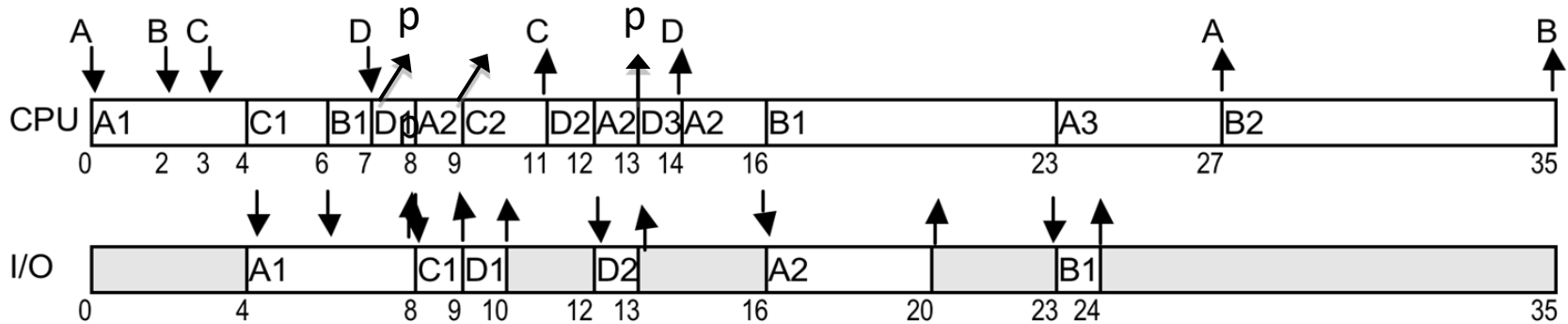
T=0	T=2	T=3	T=4	T=6	T=7	T=8	T=9	T=11	T=12	T=13	T=14	T=16	T=20	T=23	T=24	T=27
A	A ²	A ¹	B ⁸	B ⁸	B ⁷	B ⁷	B ⁷	B ⁷	B ⁷	B ⁷	B ⁷	B ⁷	B ³	A ⁴	A ³	B ² 8
	B ⁸	B ⁸	C ²	C ⁰	D ¹	A ⁴	A ³	A ³	A ³	A ²	A ²	A ⁰	A ⁴		B ⁸	
		C ²				D ⁰	C ²	D ¹		D ¹	D ⁰					

SRTF (temps restant le plus court d'abord)



- Processor utilization(occupation de processus) = $(35 / 35) * 100 = 100 \%$
- Throughput(débit) = $4 / 35 = 0.11$

SRTF (temps restant le plus court d'abord)



- Turn around time:

$$tat_A = 27 - 0 = 27$$

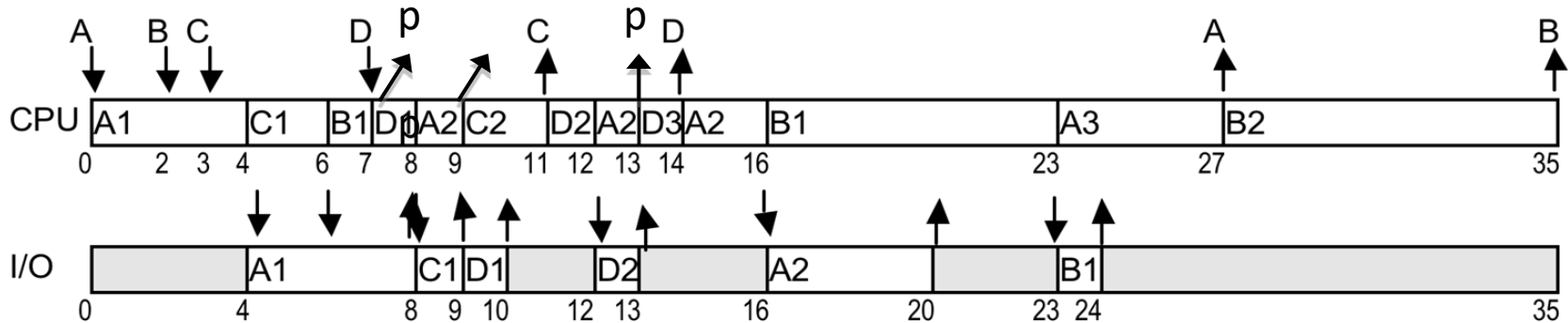
$$tat_B = 35 - 2 = 33$$

$$tat_C = 11 - 3 = 8$$

$$tat_D = 14 - 7 = 7$$

$$tat_{AVG} = (27 + 33 + 8 + 7) / 4 = 18.75$$

SRTF (temps restant le plus court d'abord)



- Waiting time:

$$wt_A = (0 - 0) + (8 - 8) + (12 - 9) + (14 - 13) + (23 - 20) = 7$$

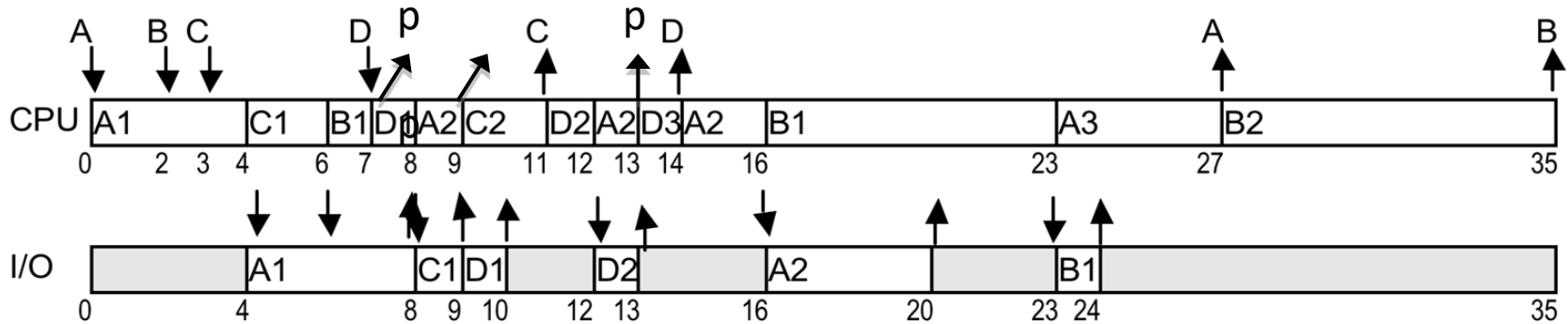
$$wt_B = (6 - 2) + (16 - 7) + (27 - 24) = 16$$

$$wt_C = (4 - 3) + (9 - 9) = 1$$

$$wt_D = (7 - 7) + (11 - 10) + (13 - 13) = 1$$

$$wt_{AVG} = (7 + 16 + 1 + 1) / 4 = 6.25$$

SRTF (temps restant le plus court d'abord)



- Response time:

$$rt_A = 0 - 0 = 0$$

$$rt_B = 6 - 2 = 4$$

$$rt_C = 4 - 3 = 1$$

$$rt_D = 7 - 7 = 0$$

$$rt_{AVG} = (0 + 4 + 1 + 0) / 4 = 1.25$$

Algorithme du Tourniquet (Round Robin)

- Algorithme conçu spécialement pour le temps partagé
- Les processus accèdent au processeur, chacun à leur tour, pour un temps maximal déterminé à l'avance (le quantum noté q = en général 10-100 millisecondes),
- Lorsqu'il a épuisé ce temps, ou qu'il se bloque : le processus suivant est élu et le remplace.
- Le processus suspendu est mis en queue du tourniquet (file FIFO circulaire).

Algorithme du Tourniquet (Round Robin)

- Si plus d'un processus essayent d'entrer en même temps à la file d'attente des processus prêts, la priorité est donnée comme suit:
 - 1: processus qui viennent d'être soumis
 - 2: processus qui ont fini une opération d'E/s
 - 3: Les processus qui ont été préemptés par le CPU

Algorithme du Tourniquet (Round Robin)

Le tourniquet (round-robin): On exécute les processus à tour de rôle.

On considère l'ensemble des processus suivant avec un **Quantum de temps = 5 UT**

Processus	Temps d'exécution
P1	12 7 2
P2	8 3
P3	4
P4	10 5
P5	5

Temps d'attente

$$P1 = 0 + (24 - 5) + (37 - 29) = 27$$

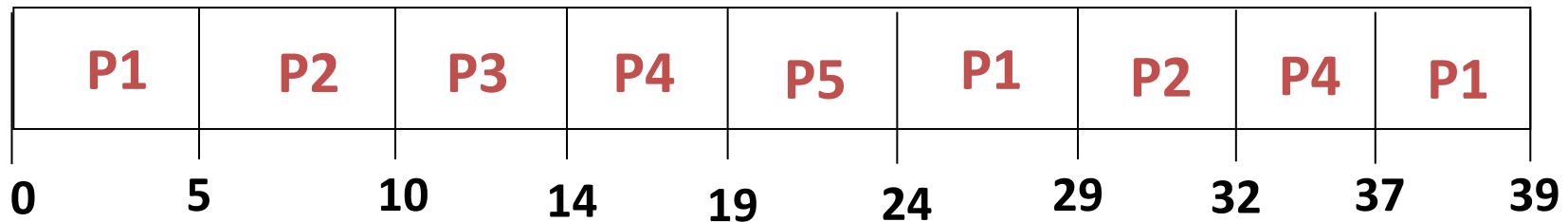
$$P2 = 5 + (29 - 10) = 24$$

$$P3 = 10$$

$$P4 = 14 + (32 - 19) = 27$$

$$P5 = 19$$

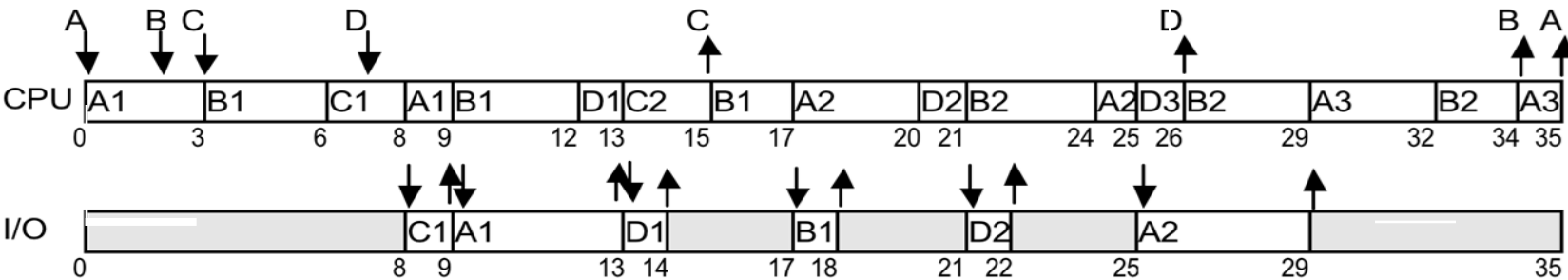
Le diagramme de Gantt pour l'ordonnancement RR avec $q = 5$ est comme suit :



$$\text{Temps d'attente moyen} = (27 + 24 + 10 + 27 + 19) / 5 = 107 / 5 = 21.4$$

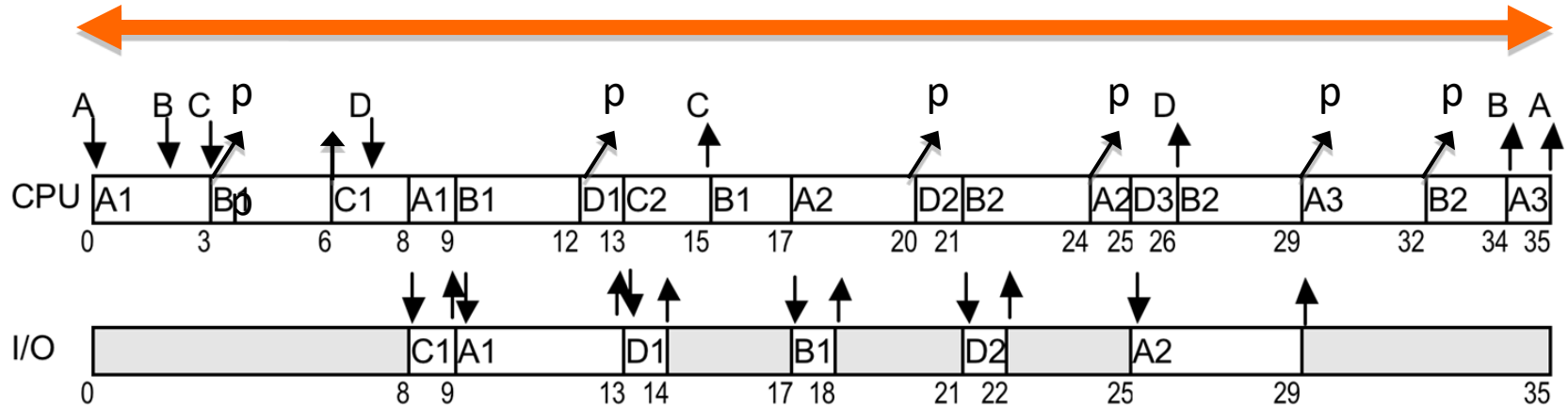
Algorithme du Tourniquet (Round

Process	Arrival time	1 st exec	1 st I/O	2 nd exec	2 nd I/O	3 rd exec
A	0	4	4	4	4	4
B	2	8	1	8	-	-
C	3	2	1	2	-	-
D	7	1	1	1	1	1



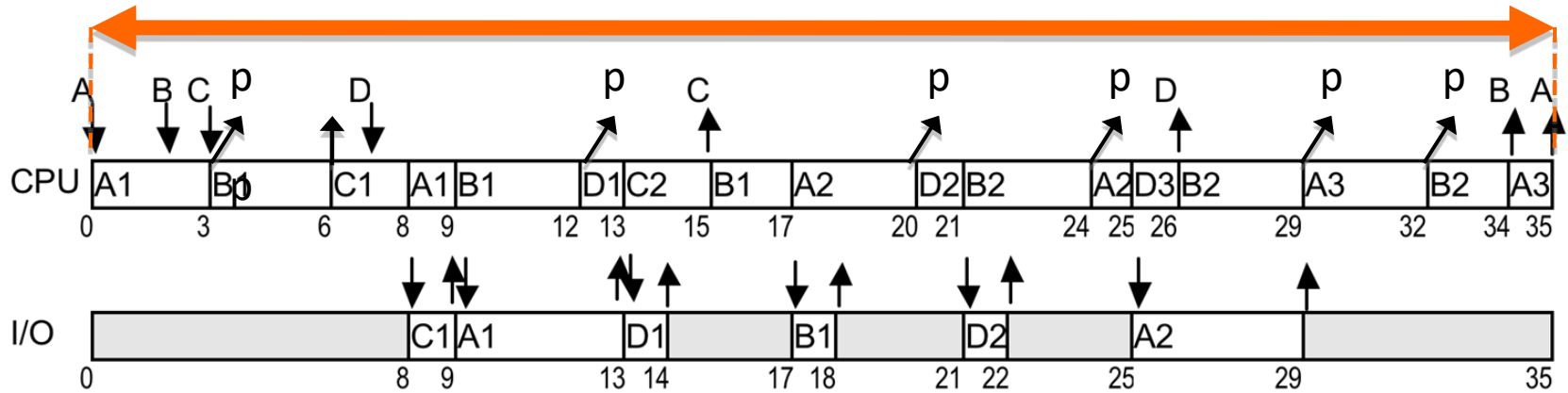
T=0	T=2	T=3	T=6	T=8	T=9	T=12	T=13	T=15	T=17	T=18	T=20	T=21	T=22	T=24	T=25
A	B	B	C	A	B	D	C	B	A	D	D	B	A	A	D
		C	A	B	D	C	B	A	D	B	B	A	D	D	B
		A	B	D	C	B	A	D			A			B	

Algorithme du Tourniquet (Round Robin)



- Processor utilization = $(35 / 35) * 100 = 100 \%$
- Throughput = $4 / 35$

Algorithme du Tourniquet (Round Robin)



- Turn around time:

$$tat_A = 35 - 0 = 35$$

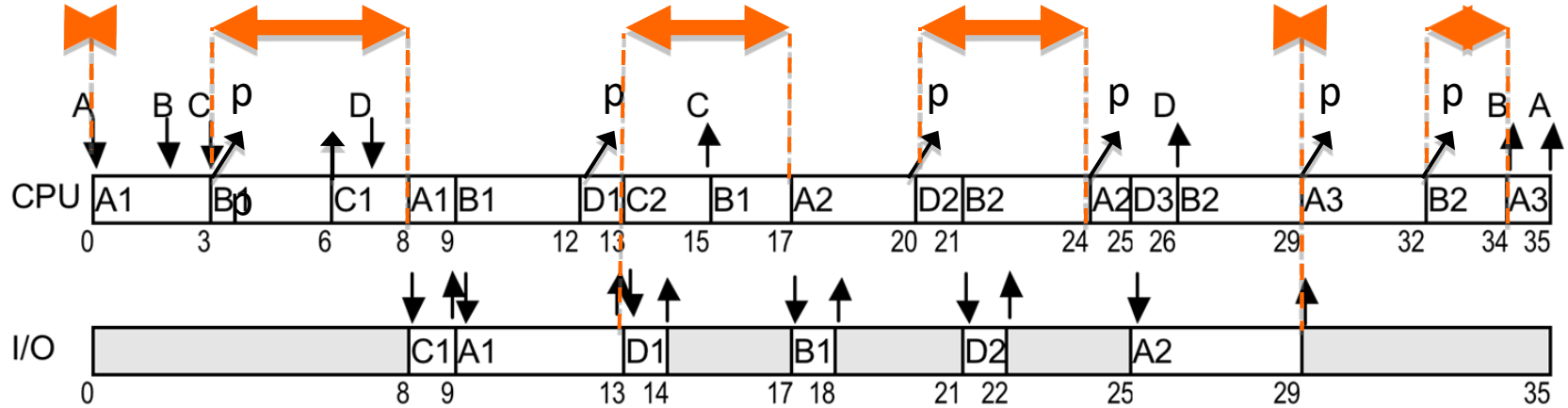
$$tat_B = 34 - 2 = 32$$

$$tat_C = 15 - 3 = 12$$

$$tat_D = 26 - 7 = 19$$

$$tat_{AVG} = (35 + 32 + 12 + 19) / 4 = 24.$$

Algorithme du Tourniquet (Round Robin)



- Waiting time:

$$wt_A = (0 - 0) + (8 - 3) + (17 - 13) + (24 - 20) + (29 - 29) + (34 - 32) = 15$$

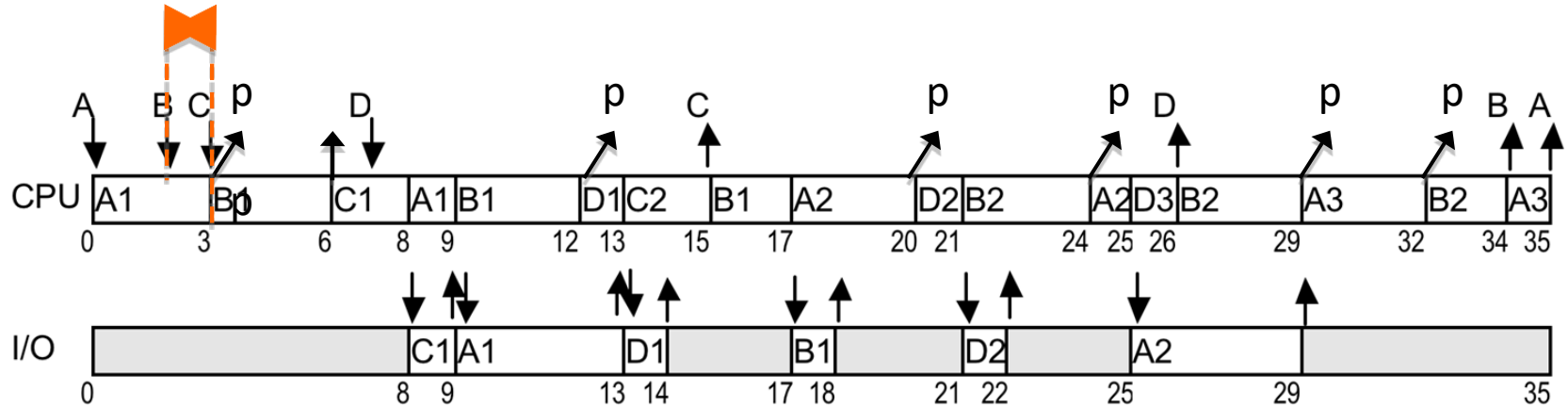
$$wt_B = (3 - 2) + (9 - 6) + (15 - 12) + (21 - 18) + (26 - 24) + (32 - 29) = 15$$

$$wt_C = (6 - 3) + (13 - 9) = 7$$

$$wt_D = (12 - 7) + (20 - 14) + (25 - 22) = 14$$

$$wt_{AVG} = (15 + 12 + 7 + 11) / 4 = 11.25$$

Algorithme du Tourniquet (Round Robin)



- Response time:

$$rt_A = 0 - 0 = 0$$

$$rt_B = 3 - 2 = 1$$

$$rt_C = 6 - 3 = 3$$

$$rt_D = 12 - 7 = 5$$

$$rt_{AVG} = (0 + 1 + 3 + 5) / 4 = 2.25$$

Ordonnancement avec priorité

- Le système d'exploitation ordonne les processus prêts selon l'ordre décroissant de leurs priorités et le processus à élire est celui avec la plus haute priorité
- Un processus ne peut s'exécuter que si aucun processus de priorité supérieure n'est dans l'état *ready*.
- Tous les processus ont la même priorité c'est la politique FIFO qui est appliquée.
- Ordonnancement
 - non-préemptif ou *préemptif*

Ordonancement avec priorité non-préemptif

Priorité: On exécute les processus selon leur priorité.

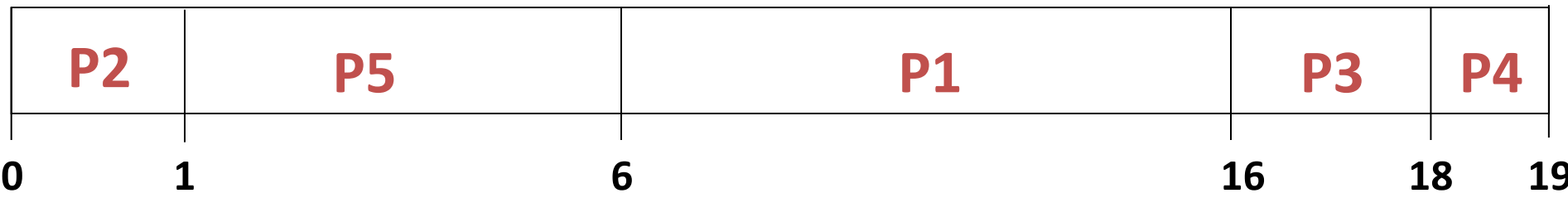
On considère l'ensemble de processus suivant :

Processus	Temps d'exécution	Priorité	Temps d'arrivée
P1	10	3	Tous les Processus Arrivent au meme temps
P2	1	5	
P3	2	2	
P4	1	1	
P5	5	4	

Temps d'attente :
début d'exécution – temps d'arrivée

P1 = 6
P2 = 0
P3 = 16
P4 = 18
P5 = 1

Le diagramme de Gantt avec un ordonnancement avec priorité est comme suit :



Temps d'attente moyen = $(6 + 0 + 16 + 18 + 1) / 5 = 41 / 5 = 8.2$

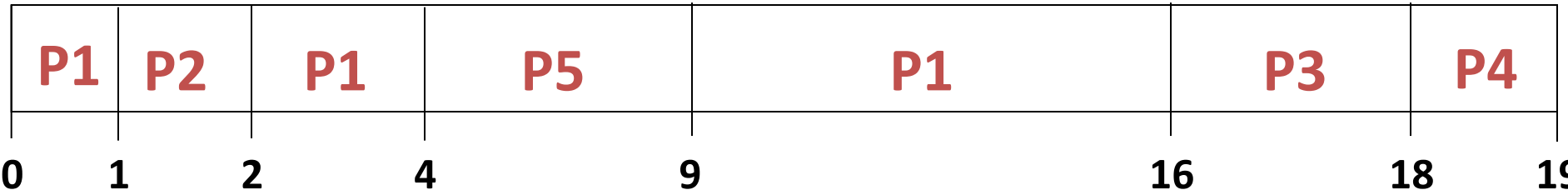
Ordonnancement avec priorité préemptif

On considère l'ensemble de processus suivant :

Processus	Temps d'exécution	Priorité	Temps d'arrivée
P1	10	3	0.0
P2	1	5	1.0
P3	2	2	2.0
P4	1	1	3.0
P5	5	4	4.0

Temps d'attente :
début d'exécution – temps d'arrivée
 $P1 = (0 - 0) + (2 - 1) + (9 - 4) = 6$
 $P2 = 1 - 1 = 0$
 $P3 = 16 - 2 = 14$
 $P4 = 18 - 3 = 15$
 $P5 = 4 - 4 = 0$

Le diagramme de Gantt avec un ordonnancement avec priorité préemptif est comme suit :



Temps d'attente moyen= $(6 + 0 + 14 + 15 + 0) / 5 = 35 / 5 = 7$

Tableau récapitulatif

	FCFS	SJF	PSJF	Tourniquet
tat_{avg}	28.25	23.50	18.75	24.50
wt_{avg}	16.50	10.50	6.25	12.25
rt_{avg}	4.50	3.00	1.25	2.25
	Facile à implémenter	Impossible de connaître le prochain temps de travail du CPU	Impossible de connaître le prochain temps de travail du CPU	Implémentable, rt_{max} est important pour les systèmes interactifs
		exactement.	exactement.	

Exercices

Exercice 1 :

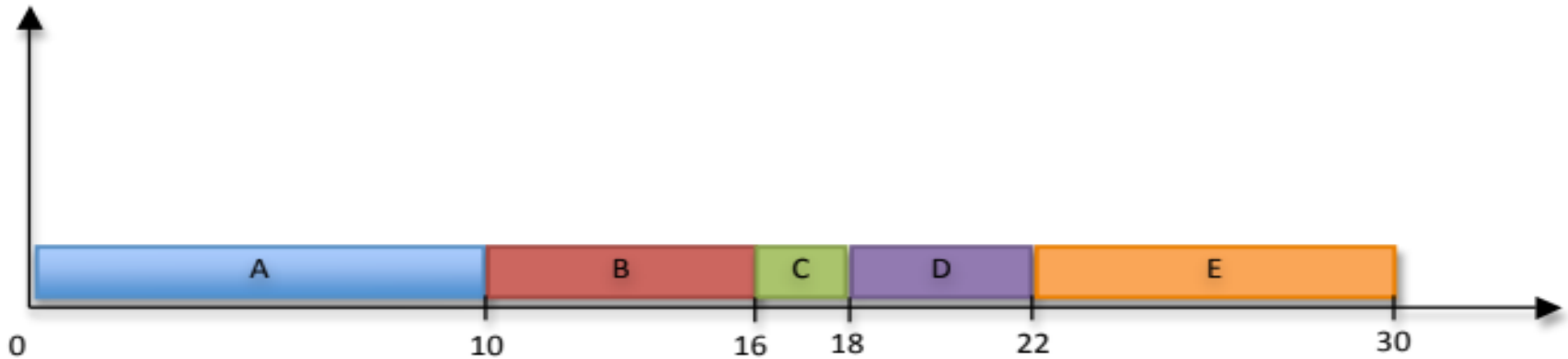
Cinq travaux A, B, C, D et E arrivent pratiquement en même temps dans un centre de calcul. Leur temps d'exécution respectif est estimé à 10, 6, 2, 4 et 8 secondes.

Tracez le digramme de Gantt et déterminez le temps moyen de rotation pour chacun des algorithmes d'ordonnancement suivants. Ne tenez pas compte du temps perdu lors de la commutation des processus.

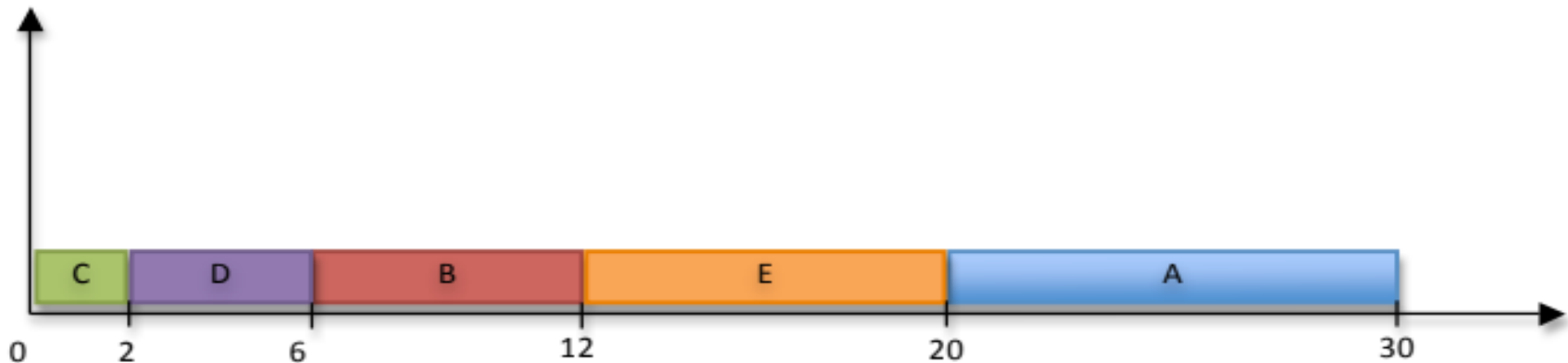
- **Premier arrivé, premier servi FCFS (exécution dans l'ordre 10, 6, 2, 4, 8) ;**
- **Plus court d'abord SJF ;**
- **Tourniquet (quantum $q = 4$ s).**

SOLUTION

- Algorithme FCFS $TRM = ((10-0)+(16-0)+(18-0)+(22-0)+(30-0)) / 5 = 19,2$

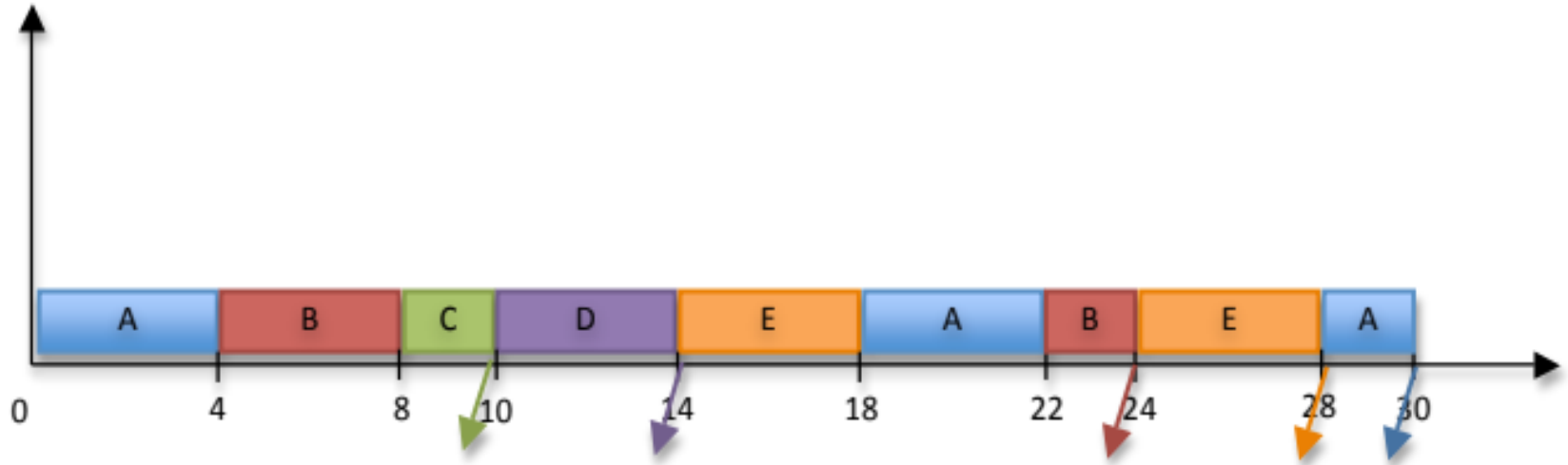


- Algorithme SJF $TRM = ((2-0)+(6-0)+(12-0)+(20-0)+(30-0)) / 5 = 14$



SOLUTION

- Algorithme RR $TRM = ((10-0)+(14-0)+(24-0)+(28-0)+(30-0)) / 5 = 21,2$



Exercices

Exercice 2 :

On considère l'ensemble des processus suivants :

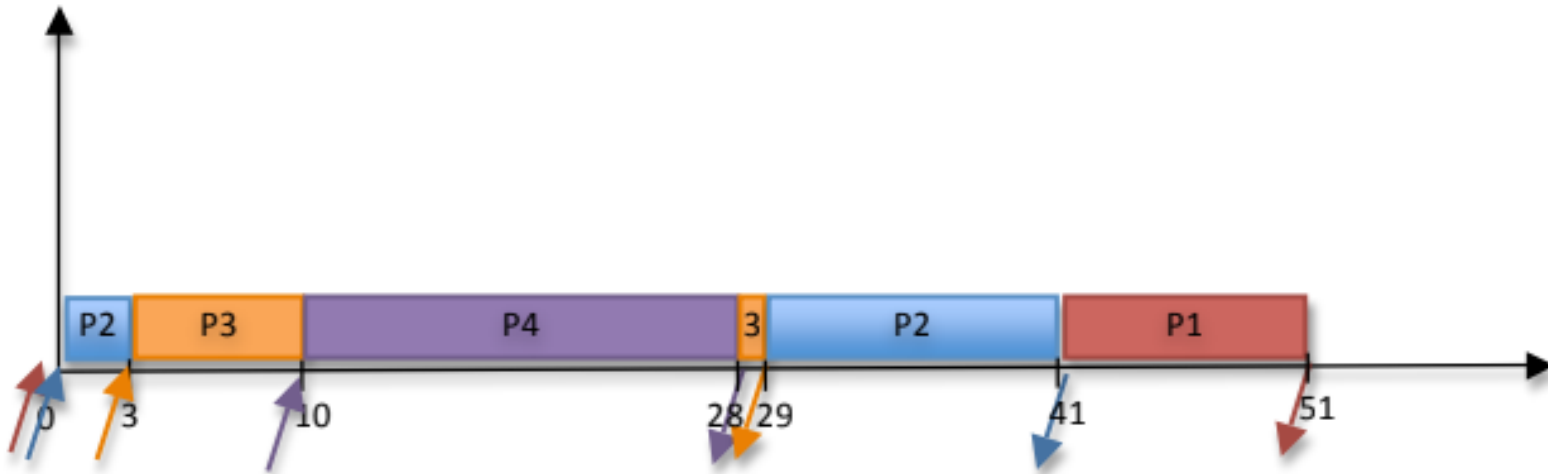
Processus	Date d 'arrivée	Temps CPU	Priorité
1	7h00	10 mn	2
2	7h00	15 mn	3
3	7h03	8 mn	4
4	7h10	18 mn	5

**A- On suppose qu'on utilise un algorithme d'ordonnancement basé sur la priorité (les priorités sont décroissantes : 5 est le plus prioritaire).
Donnez le diagramme de Gantt pour les priorités données dans le tableau.**

SOLUTION

Priorité Statique

- **TRM** = $((28-10)+(29-3)+(41-0)+(51-0)) / 4 = 34$
- **TAM** = $((18-18)+(26-8)+(41-15)+(51-10)) / 4 = 21,25$



Exercices

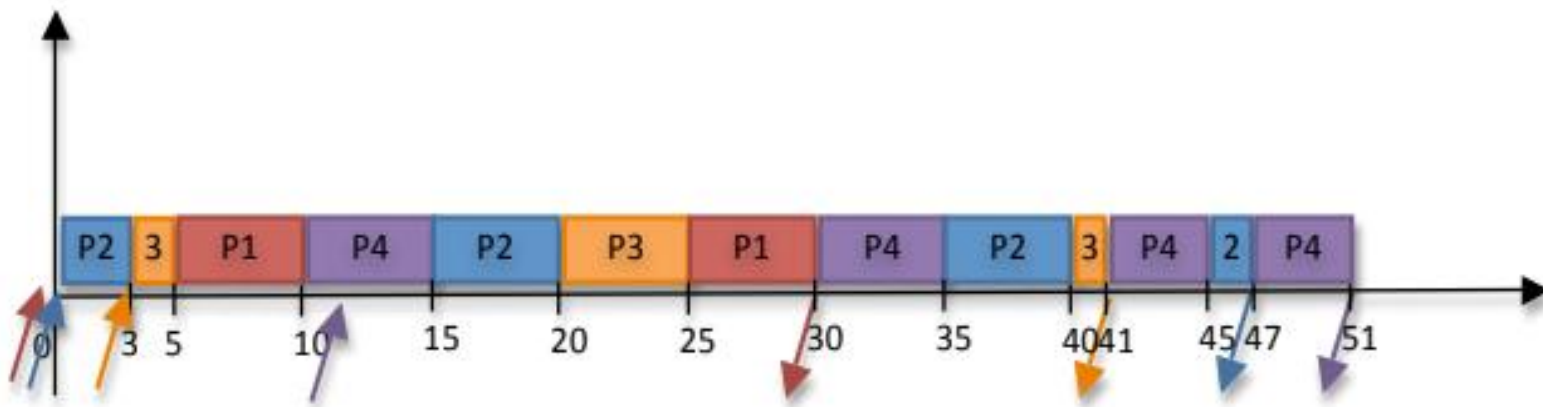
B- On voudrait que la priorité des processus soit dynamique au cours du temps. Ainsi, pour calculer la priorité d'un processus, on utilise la formule suivante :

$$\text{Priorité} = \frac{\text{Temps d'attente} + \text{Temps CPU Restant}}{\text{Temps CPU}}$$

Remarque : Lors des calculs, on arrondira suivant l'exemple suivant : 3.5 ou 3.6 -> 4, 3.1 ou 3.4 -> 3.

- 1. Donnez le diagramme de Gantt sachant que la priorité est recalculée toutes les 5 minutes.**
- 2. Calculez le temps d'attente moyen ainsi que le temps de rotation moyen.**
- 3. Comparez les résultats obtenus par rapport à ceux obtenus avec l'algorithme de priorité classique.**

SOLUTION



	0	3	5	10	15	20	25	30	35	40	41	45
P₁	2	2	$5+10/10=2$	$0+5/10=1$	$5+5/10=1$	$10+5/10=2$	$15+5/10=2$					
P₂	3	3	$2+12/15=1$	$7+12/15=1$	$12+12/15=2$	$0+7/15=0$	$5+7/15=1$	$10+7/15=1$	$15+7/15=1$	$0+2/15=0$	0	$5+2/15=0$
P₃		4	$0+6/8=1$	$5+6/8=1$	$10+6/8=2$	$15+6/8=3$	$0+1/8=0$	$5+1/8=1$	$10+1/8=1$	$15+1/8=2$		
P₄				5	$0+13/18=1$	$5+13/18=1$	$10+13/18=1$	$15+13/18=2$	$0+8/18=0$	$5+8/18=1$	1	$0+4/18=0$

SOLUTION

Priorité Dynamique

- $\text{TRM} = ((30-0)+(47-0)+(41-3)+(51-10)) / 4 = 39$
- $\text{TAM} = ((30-0-10)+(47-0-15)+(41-3-8)+(51-10-18)) / 4 = 26,25$

Chapitre3 : Les Processus : Création et synchronisation

Les processus

- Processus:
 - Instance d'un programme en exécution
 - et un ensemble de données
 - et un ensemble d'informations (BCP, bloc de contrôle de processus)
- Deux types de processus
 - Système: propriété du super-utilisateur (démons)
 - Utilisateur: lancés par les utilisateurs
- Code de retour d'un processus
 - **=0**: fin normale
 - **≠0**: comportement anormal (en cours d'exécution ou à la terminaison)
- Interruption d'un processus à la réception d'un signal
 - A partir d'un programme (SIGINT, SIGQUIT)
 - A partir d'un shell avec la commande:
`kill num_signal pid_processus`

Différence entre programme et processus

Programme

- Entité statique décrivant un traitement ;
- Code situé sur disque (en langage source, en langage machine)
- Un programme peut donner lieu à plusieurs processus possibles par exemple : un même programme exécuté avec des données différentes

Processus

- Entité dynamique réalisant un traitement ;
- Code situé en mémoire centrale (en langage machine) ;
- Un processus peut mettre en jeu plusieurs programmes par exemple : un programme se terminant avec le lancement d'un autre programme (recouvrement).

Les processus

Identification : le PID

- Identification de processus : **le PID** (process id)
- `pid_t getpid(void)`; Appel système retournant le PID du processus
- visibles par les commandes `top`, `ps`
- Nom de l'entrée dans le dossier `/proc`

Affiliation

- tout processus à un père : processus qui l'a lancé
- `pid_t getppid(void)`; Appel système retournant le PID du processus père
- L'ancêtre `pid = 1`, le processus `init`, lancé au boot
- les orphelins (processus dont le père est mort) sont le plus souvent récupérés par `init`

Etats d'un processus

Sortie d'une commande ps

<i>D</i>	sommeil ininterrompible
<i>R</i>	Actif ou prêt (dans la file)
<i>S</i>	Sommeil interrompible (attente d'un événement)
<i>T</i>	Stoppé (par un signal)
<i>X</i>	Mort
<i>Z</i>	Deficient ("zombie")
	processus, terminé mais données non recup par parent

Caractères additionnels

<i><</i>	Processus ayant une très haute priorité sur les autres
<i>N</i>	Basse priorité
<i>s</i>	session leader
<i>l</i>	multi-threadé
<i>+</i>	En avant plan

Informations de la commande *ps*

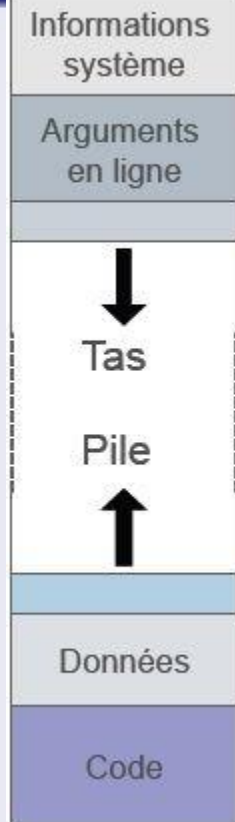
Nom	Interprétation	Option(s)
S	Etat du processus S: endormi, R: actif, T: stoppé, Z: terminé (zombi)	-l
F	Indicateur de l'état du processus en mémoire (swappé, en mémoire, ...)	-l
PPID	Identificateur du processus père	-l -f
UID	Utilisateur propriétaire	-l -f
PGID	N° du groupe de processus	-j
SID	N° de session	-j
ADDR	Adresse du processus	-l
SZ	Taille du processus (en nombre de pages)	-l
WCHAN	Adresse d'événements attendus (raison de sa mise en sommeil s'il est endormi)	-l
STIME	Date de création	-l
CLS	Classe de priorité (TS → temps partagé, SYS → système, RT → temps réel).	-cf -cl
C	Utilisation du processeur par le processus	-f -l
PRI	Priorité du processus	-l
NI	Valeur « nice »	-l

Image

Ensemble des composants d'une image

Un programme en cours d'exécution manipule (met en jeu)

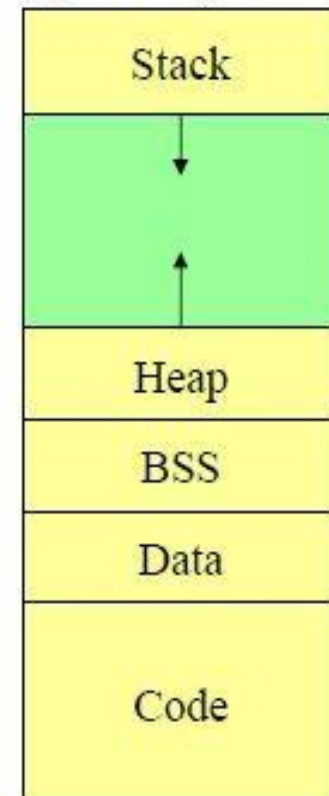
- Code
- Données
 - ▶ Statiques
 - ▶ Tas
 - ▶ Pile
- Contexte d'exécution
 - ▶ Pointeur d'instruction
 - ▶ Registres mémoire
 - ▶ Fichiers ouverts
 - ▶ Répertoire courant
 - ▶ Priorité



Processus en mémoire

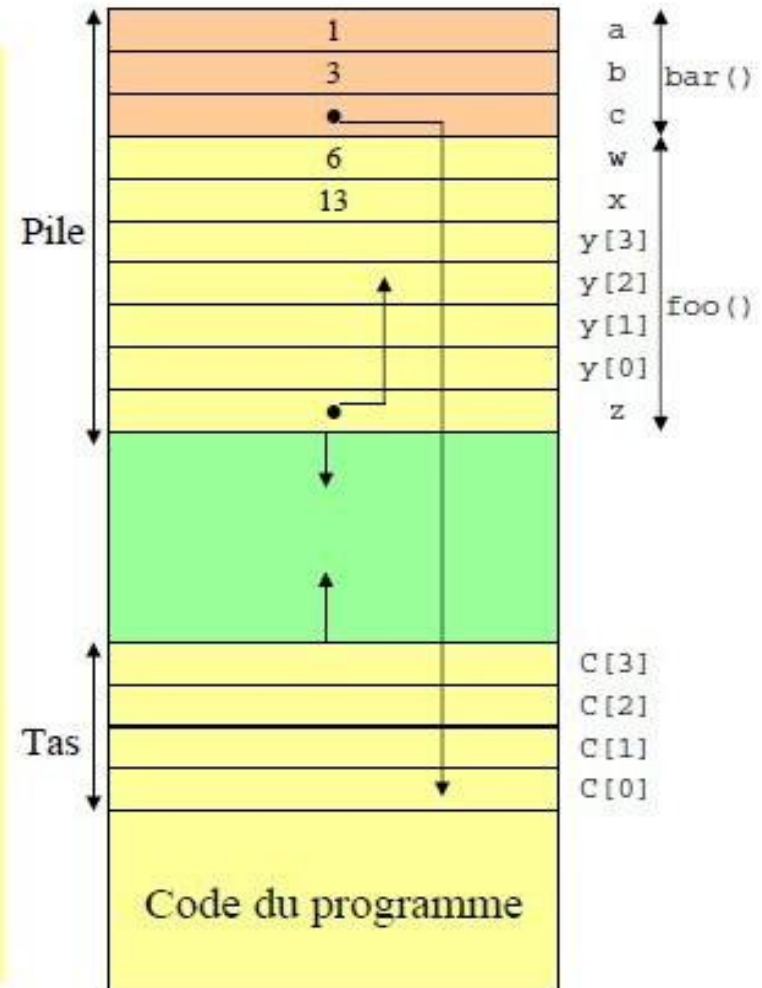
- Pile (stack):
 - Les appels de fonctions et variables locales aux fonctions exigent une structure de donnée dynamique
 - Variables locales et paramètres
 - Adresse de retour de fonction
- Tas (heap)
 - Allocation dynamiques de variables
- Code (Text)
 - Code des fonctions
 - Lecture seulement (read-only)
- Données (Data+BSS)
 - Constantes
 - Variables initialisées / non-initialisées

Image d'un processus

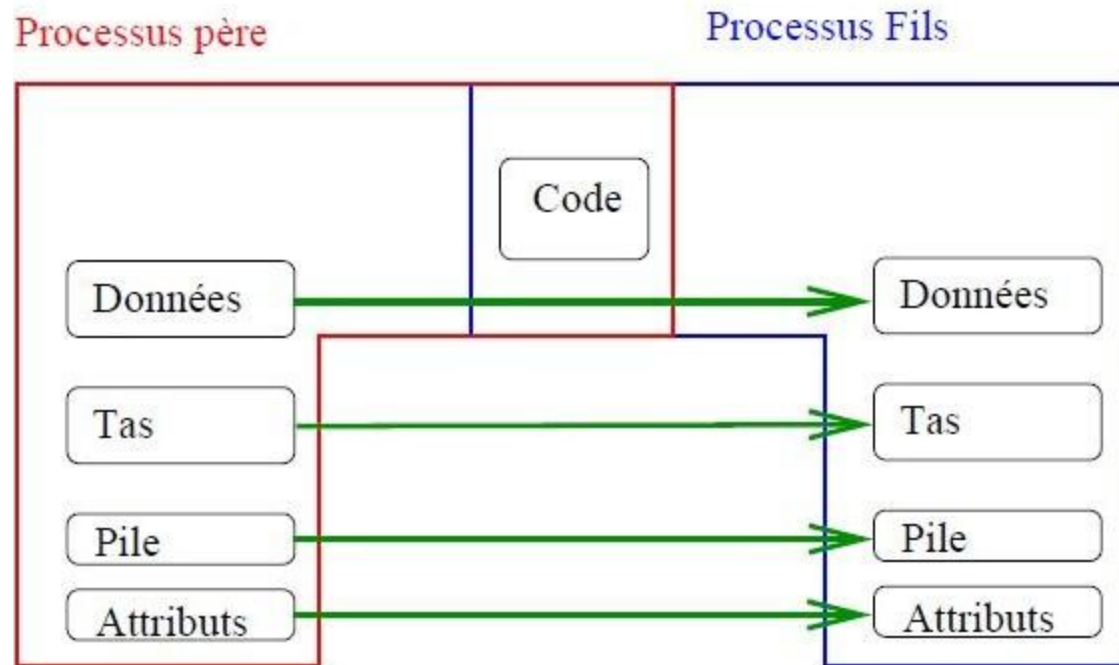


Processus en mémoire

```
void foo(int w) {  
    int x = w+7;  
    int y[4];  
    int *z = &y[2];  
}  
  
void bar(int a) {  
    int b = a+2;  
    int *c = (int *) calloc(sizeof(int)*4);  
    foo(b+3);  
}  
  
main() {  
    bar(1);  
}
```



Création de processus Clonage



Création de processus

Primitive `fork()`, clônage (Sous Unix)

- Primitive système : `fork()`
- Recopie totale (données, attributs) du processus père vers son processus fils (nouveau pid).
- Le fils continue son exécution à partir de cette primitive

Code C : `p1.c`

```
#include<stdlib.h>
int main() {
    printf("Coucou %d\n",getpid());
    fork();
    printf("Hé %d\n",getpid());
}
```

Résultat

```
Machine:~/Moi> ./p1
Coucou 6874
Hé 6875
Hé 6874
Machine:~/Moi>
```

Création de processus

Pour créer un processus, le système doit :

- Nommer le processus
- Créer un bloc de contrôle BCP
- Déterminer la priorité du processus
- Allouer des ressources au processus

Ensuite

- Recopie des données
- Recopie des attributs sauf
 - ▶ pid, ppid
 - ▶ signaux pendants
 - ▶ priorité
- Partage du code (si réentrance) sinon recopie du code

Création de processus Déroulement de l'exécution

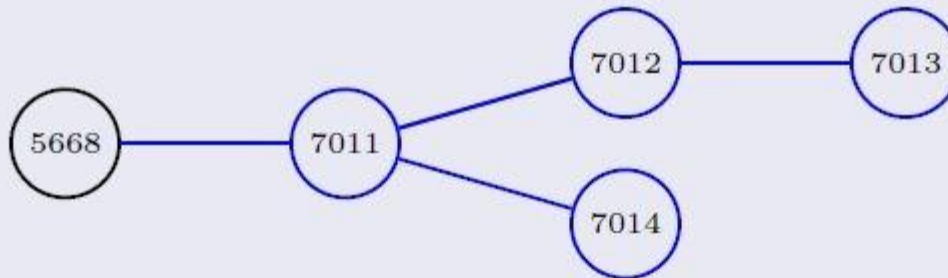
Code C : p2.c

```
#include<unistd.h>
int main() {
    printf("Coucou %d\n",getpid());
    fork();
    fork();
    printf("Hé %d:%d\n",getpid(),
           getppid());
}
```

Résultat

```
Machine:~/Moi> ./p2
Coucou 7011
Hé 7013:7012
Hé 7012:7011
Hé 7014:7011
Hé 7011:5668
Machine:~/Moi>
```

Hiérarchie

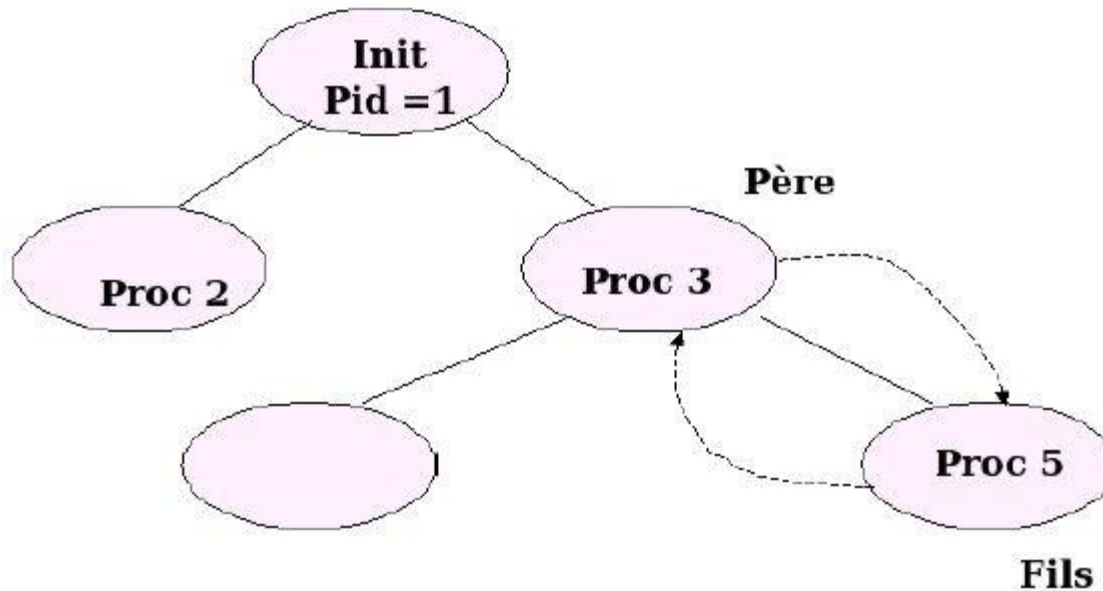


- Le processus fils hérite de beaucoup d'attributs du père mais n'hérite pas:
 - De l'identification de son père
 - Des temps d'exécution qui sont initialisés à 0
 - Des signaux pendants (arrivées, en attente d'être traités) du père
 - De la priorité du père; la sienne est initialisée à une valeur standard
 - Des verrous sur les fichiers détenus par le père
- Le fils travaille sur les données du père s'il accède seulement en lecture. S'il accède en écriture à une donnée, celle-ci est alors recopiée dans son espace local.

Création de processus

Arborescence

L'itération de `fork()` conduit à une arborescence à partir du processus init (pid =1)



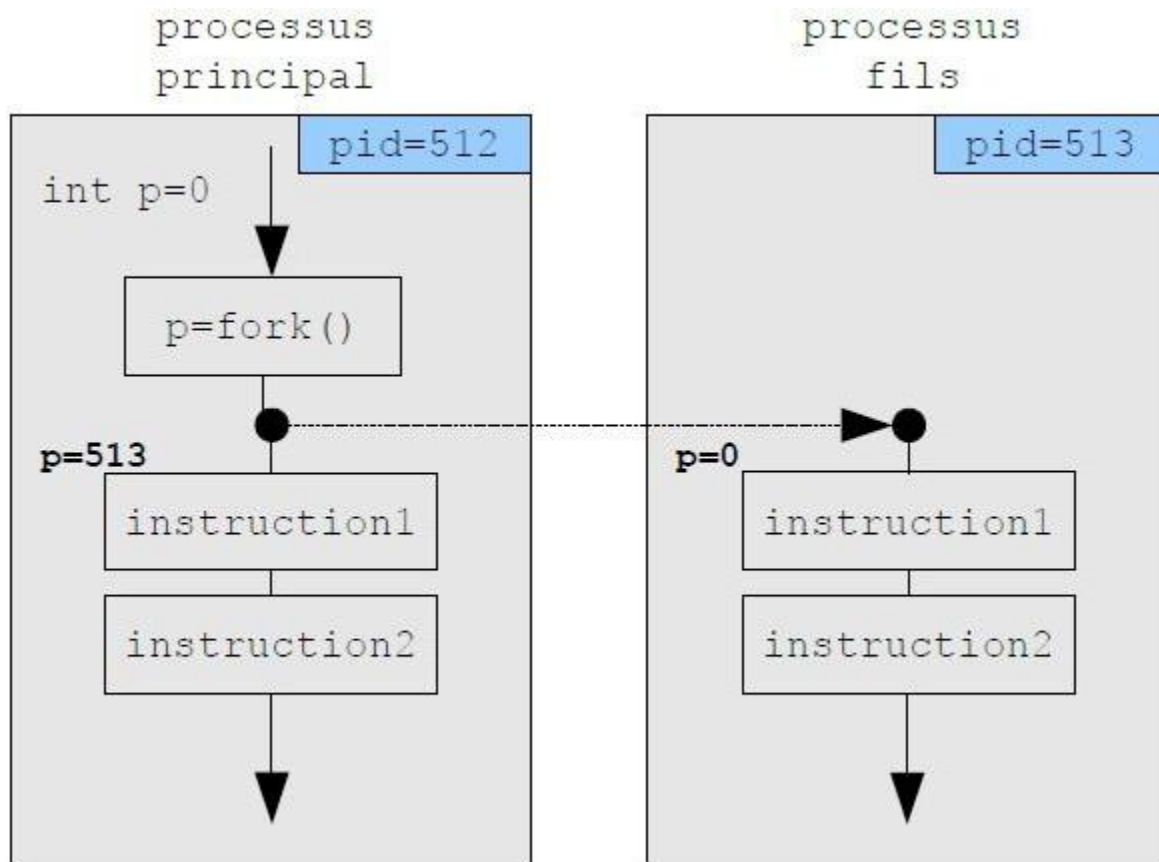
Création de processus Différence fils-père

Différencier le père du fils ?

- Code de retour du `fork()`
- Dans le père : le `fork` retourne le pid du processus fils
- Dans le fils : le `fork` retourne 0

Pourquoi ?

- Le fils peut connaître le pid de son père avec `getppid()`.
- Le code de retour du `fork` : seul moyen pour le père de connaître le pid du processus fils.



Int fork() :

- ❑ Crée un nouveau processus (**processus fils**) par duplication du processus courant (**processus père**)
- ❑ Copie les segments de données et de stack du processus père; le segment texte est partagé.
- ❑ Les ``pid" du père et du fils diffèrent
- ❑ Le fils hérite en partie de l'environnement du père.
- ❑ Les processus père et fils poursuivent l'exécution du **même programme**.
- ❑ La valeur retournée par **fork** permet de distinguer entre **le père** et **le fils**.
- ❑ Le **fils** continue son exécution a partir de cette primitives.

```
#include <stdio.h>
#include <unistd.h>

int main(void){
    int pid;

    pid = fork();

    if (pid > 0)
        printf("processus père: %d-%d-%d\n", pid, getpid(), getppid());

    if (pid == 0) {
        printf("processus fils: %d-%d-%d\n", pid, getpid(), getppid());
    }

    if (pid < 0)
        printf("Probleme de creation par fork()\n");

    system("ps -l");
    return 0;
}
```


Code C : p3.c

```
#include<unistd.h>
int main() { int r;
    printf("Coucou %d\n",getpid());
    if ( (r=fork())==0 )
        printf("Fils %d\n",getpid());
    else
        printf("Père %d\n",getpid());
    printf("Tous %d\n",getpid());
}
```

Résultat

```
Machine:~/Moi> ./p3
Coucou 7209
Père 7209
Tous 7209
Fils 7210
Tous 7210
Machine:~/Moi>
```


Portée du code : p4.c

```
int main(){
    int k ;
    printf(''Je suis seul au monde'\n');
    k=fork();
    if (k==-1) { printf("Erreur fork()"); exit(1);}
    if (k== 0) { /* code du fils */
        printf("Je suis le processus fils\n");
        exit(0);
    } else
        printf("Je suis le processus pere\n");
    printf("Et la qui suis-je %d\n",getpid());
}
```

Portée du code : Résultat

```
Machine:~/Moi> ./p4
Je suis seul au monde
Je suis le processus fils 6681
Je suis le processus pere 6680
Et la qui suis-je 6680
Machine:~/Moi>
```

Portée des variables : p5.c

```
int main(){
    int i,j,k ;
    i=5; j=2;
    if ((k=fork()) == -1) {printf("Erreur fork()");exit(1);}
    if (k== 0) { /* code du fils */
        printf("fils %d \n",getpid());
        j--;
    } else {
        printf("pere %d\n",getpid());
        i++;
    }
    printf("Pid:%d i:%d j:%d\n", getpid(),i,j);
}
```

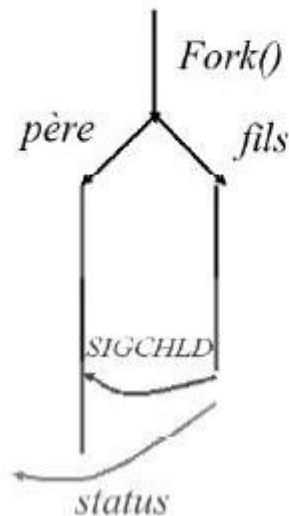
Portée des variables : Résultat de p6

```
Machine:~/Moi> ./p5
fils 6774
Pid:6774 i:5 j:1
pere 6773
Pid:6773 i:6 j:2
Machine:~/Moi>
```

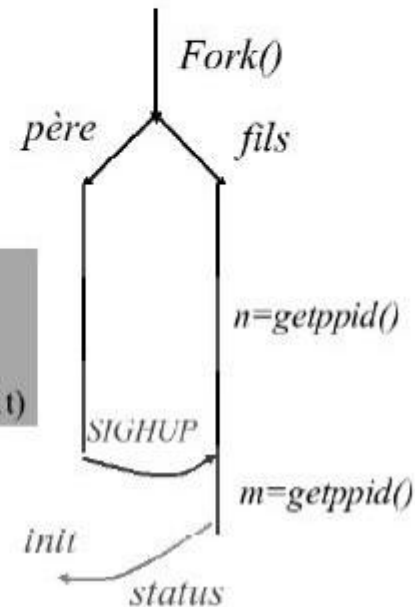
Indéterminisme du déroulement

Exécutions concurrentes

- Parallélisme d'exécution du père et du fils
- Concurrence pour l'accès aux ressources
- Ordre du déroulement (et des fins) est impossible à contrôler



Père se termine
avant le fils
 $n = \text{pid}$ du père
 $m = 1$ (pid de l'init)



Terminaison de processus

- Un processus se termine par une demande d'arrêt volontaire (**exit**) ou par un arrêt forcé provoqué par un autre processus (appel système **kill**) ou une erreur.

`void exit(int vstatus);`

- Lorsqu'un processus fils se termine :
 - son état de terminaison est enregistré dans son PCB,
 - la plupart des autres ressources allouées au processus sont libérées,
 - le processus passe à l'état zombie (<defunct>).
- Son PCB et son PID sont conservés jusqu'à ce que son processus père ait récupéré cet état de terminaison. Il est alors détruit.
- Les appels système `wait(&status)` et `waitpid(pid, &status, option)` permettent au processus père de récupérer, dans `status`, cet état de terminaison.

Synchronisation

- Père toujours prévenu de la fin d'un fils
- Fils toujours prévenu fin du père
- mais il faut que le père soit en attente
- Si la fin d'un fils n'est pas traitée par le père ce processus devient un processus **zombie**.

Synchronisation pour les fins d'exécution

- Possibilités d'échanges d'informations permettant une synchronisation sur les fins d'exécutions.
- Éviter les processus *zombies* (fin propre).
- Nécessite que le père soit en attente.
- Information de la fin du processus fils.

Appels système wait et exit

exit et code de retour

`exit (int)`

- valeur du `int` est “transmise” au père : **code de retour**
- Fin du processus fils après `exit`
- Par convention (défaut) une fin correcte donne un code de retour nul.

wait

`int wait(int *)`

- Entier retourné : pid du fils qui s'est terminé depuis l'invocation du `wait`
- Si aucun fils susceptible de se terminer alors renvoi de `-1`
- L'entier pointé enregistre l'état du fils lorsqu'il se finit (valeur en paramètre dans `exit`)

Appels système wait et exit

```
#include <sys/types.h> #include <sys/wait.h>
pid_t wait(int *etat);
pid_t waitpid(pid_t pid, int *etat, int options);
```

- Ces deux primitives permettent l'élimination des processus zombies et la synchronisation d'un processus sur la terminaison de ses descendants avec récupération des informations relatives à cette terminaison.
- Elles provoquent la suspension du processus appelant jusqu'à ce que l'un de ses processus fils se termine
- La primitive `waitpid` permet de sélectionner un processus particulier parmi les processus fils (`pid`)

Création de processus et synchronisation

```
#include <stdio.h>

int main(void)
{
    if (fork() == 0) {
        printf("Fin du processus fils de N° %d\n", getpid());
        exit(2);
    }
    sleep(30);
    wait(0);
}
```


Code C : p6.c

```
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main() { int r,s,w;
    if ( (r=fork())==0 ) {
        printf("Fils %d\n",getpid());
        // Traitement long
        exit(14);
    } else {
        // Père doit attendre la
        // mort de son fils.
        w=wait(&s);
        printf("w:%d s:%d\n",w,s);}
}
```

Résultat

```
Machine:~/Moi> ./p4
Fils 7344
w:7344 s:2560
Machine:~/Moi>
```

Recouvrement d'un processus

Nouveau code à la place d'un autre

Un processus peut changer de code par un appel système à `exec`.

- code et données remplacés
- pointeur d'instruction réinitialisé

Appel Système : `exec()`

```
#include <unistd.h>
```

```
int execl(const char *path, const char *arg0, ...,  
          const char *argn, char * /*NULL*/);
```

`path` = nom de l'exécutable recherché dans `$PATH`

`arg0` = nom exécutable affiché par `ps`

`argn` = $n - 2$ ème argument de l'exécutable

`NULL` = argument de fin de la ligne de commande

Recouvrement d'un processus

Exemple : r1.c

```
#include<unistd.h>
int main() {
    printf("Je suis un programme qui va exécuter /bin/ls -l\n");
    execl("/bin/ls","ls","-l",NULL);
}
```

Recouvrement d'un processus : Résultats

```
Machine:~/Moi/Test> ls
d1  f1  f2
Machine:~/Moi/Test> ../r1
Je suis un programme qui va exécuter /bin/ls -l
total 4
drwxr-xr-x 2 blec sys 4096 2006-09-29 14:00 d1
-rw-r--r-- 1 blec sys    0 2006-09-29 14:00 f1
-rw-r--r-- 1 blec sys    0 2006-09-29 14:00 f2
Machine:~/Moi/Test>
```

Recouvrement d'un processus

- ❶ Le programme `r1.c` exécute le `printf`
- ❷ lorsqu'il exécute le `execl`, il est totalement remplacé par l'exécution du `"ls -l"`
- ❸ le processus tel qu'il était avec le programme, disparaît

Recouvrement d'un processus : remplacement du code

Exemple : r2.c

```
#include<unistd.h>
int main() {
    printf("Je suis un programme qui va exécuter /bin/ls -l\n");
    execl("/bin/ls","ls","-l",NULL);
    printf("Je suis un programme qui a exécuté /bin/ls -l\n");
    /// Code qui ne sera pas exécuté !!!!!
}
```

Résultats

```
Machine:~/Moi/Test> ls
d1  f1
Machine:~/Moi/Test> ../r2
Je suis un programme qui va exécuter /bin/ls -l
total 4
drwxr-xr-x 2 blec aoc 4096 2006-09-29 14:00 d1
-rw-r--r-- 1 blec aoc    0 2006-09-29 14:00 f1
Machine:~/Moi/Test>
```

Création puis Recouvrement et synchronisation

Exemple : pr1.c

```
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>
int main() { int r,status,w;
    r=fork();
    if ( r==0 ) {
        execl("/bin/ls","ls","-a",NULL);
    }
    else {
        printf("Attente du fils\n");
        w=wait(&status);
        printf("w:%d s:%d\n",w,status);
    } }
```

Résultats

```
Machine:~/Moi/Test>../pr1
Attente du fils
..  d1 f1 f2 d2
w:7508 s:0
Machine:~/Moi/Test>
```

Lancement d'un binaire par le shell ??

Shell lance un processus fils puis recouvrement par la commande

Conclusion

`pid_t fork()`

fonction qui crée un processus fils par duplication du processus qui y fait appel. En cas de succès, la fonction `fork` retourne la valeur du `pid` du fils au processus père et la valeur 0 au processus fils créé. En cas d'erreur lors de la création du processus fils, cette fonction renvoie -1 au processus père et le processus fils ne sera pas créé.

`pid_t getpid()`

renvoie le `pid` du processus qui y fait appel.

`pid_t getppid()`

renvoie le `pid` du père du processus qui y fait appel.

`void exit(int status)`

provoque la terminaison du processus en cours. Il prend en argument un code de retour. Ce code peut être récupéré par le processus père (*voir wait*). Cette fonction ne renvoie pas de valeur dans la fonction qui l'invoque.

`pid_t wait(int *status)`

attend la fin d'un processus fils. `wait` retourne le `pid` du fils en cas de succès et -1 en cas d'erreur. `status` récupère la valeur retournée par le fils, par exemple par la commande `exit(status)`.

```
int exec()
```

permet de lancer et d'exécuter un programme externe en remplaçant le processus en cours. Différentes variantes pour la spécification du fichier et/ou des arguments existent : `execl`, `execle`, `execlp`, `execv`, `execvp`, ...

```
execl (char *path, char *arg0, char *arg1, ..., char *argn, NULL)
```

- cette variante exécute un programme ;
- `path` est le chemin du fichier programme ;
- `arg0` : premier argument (correspondant au nom du programme exécutable);
- `argn` est le `n` ième argument.

Exemple : `execl ("/bin/ls", "ls", "-la", NULL) ;`

- exécute la commande `ls`
- située dans le répertoire `/bin/`
- en utilisant le paramètre `-la`


```
# include <unistd.h>
# include <stdio.h>
# include <sys/types.h>
int main ( )
{
pid_t fils_pid ;
fils_pid=fork( );
if (fils_pid ==0)
printf ( " Je suis le fils avec pid %d -- %d \n" , getpid(), getppid()) ;
else if ( fils_pid > 0 )
printf ( "Je suis le pere avec pid  %d  \n" , getpid() );
else
printf ( "Erreur dans la creation du fils \n" ) ;
}
```

```
Je suis le pere avec pid  9622
```

```
Je suis le pere avec pid  4344
Je suis le fils avec pid 4345 -- 1
```

```
Je suis le pere avec pid  10479
Je suis le fils avec pid 10480 -- 10479
```

```
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>

int main()
{
    int i;

    printf("Dans le corps du programme.\n");

    i=fork();

    printf("Après le fork, i=%d\n",i);

    return 0;
}
```

Chapitre 4: Communication interprocessus

Introduction

Les processus coopèrent souvent pour traiter un même problème. Ces processus s'exécutent en parallèle sur un même ordinateur (monoprocasseur ou multiprocesseurs) ou bien sur des ordinateurs différents. Ils doivent alors s'échanger des informations (**communication interprocessus**). Il existe plusieurs moyens de communication interprocessus. Nous pouvons citer, entre autres, les variables communes, les fichiers communs, les signaux, les messages et les tubes de communication

Les entrées-sorties

- ▶ Les lectures/écritures sont appelées entrées/sorties.
- ▶ Une entrée/sortie provoque une interruption. Le processus appelant est mis en attente (**sommeil**) le temps que sa demande soit traitée.
- ▶ Pour accélérer les entrées sorties, les systèmes utilisent un mécanisme de cache : lorsqu'un processus fait une écriture, elle est réalisée en RAM et non sur le périphérique réel, dans une zone appelée **buffer cache**.
- ▶ Le buffer cache est périodiquement **synchronisé** avec le périphérique.
- ▶ Ce mécanisme est transparent pour le programmeur, à part qu'au retour d'une écriture, les données ne sont pas nécessairement écrites sur le périphérique.

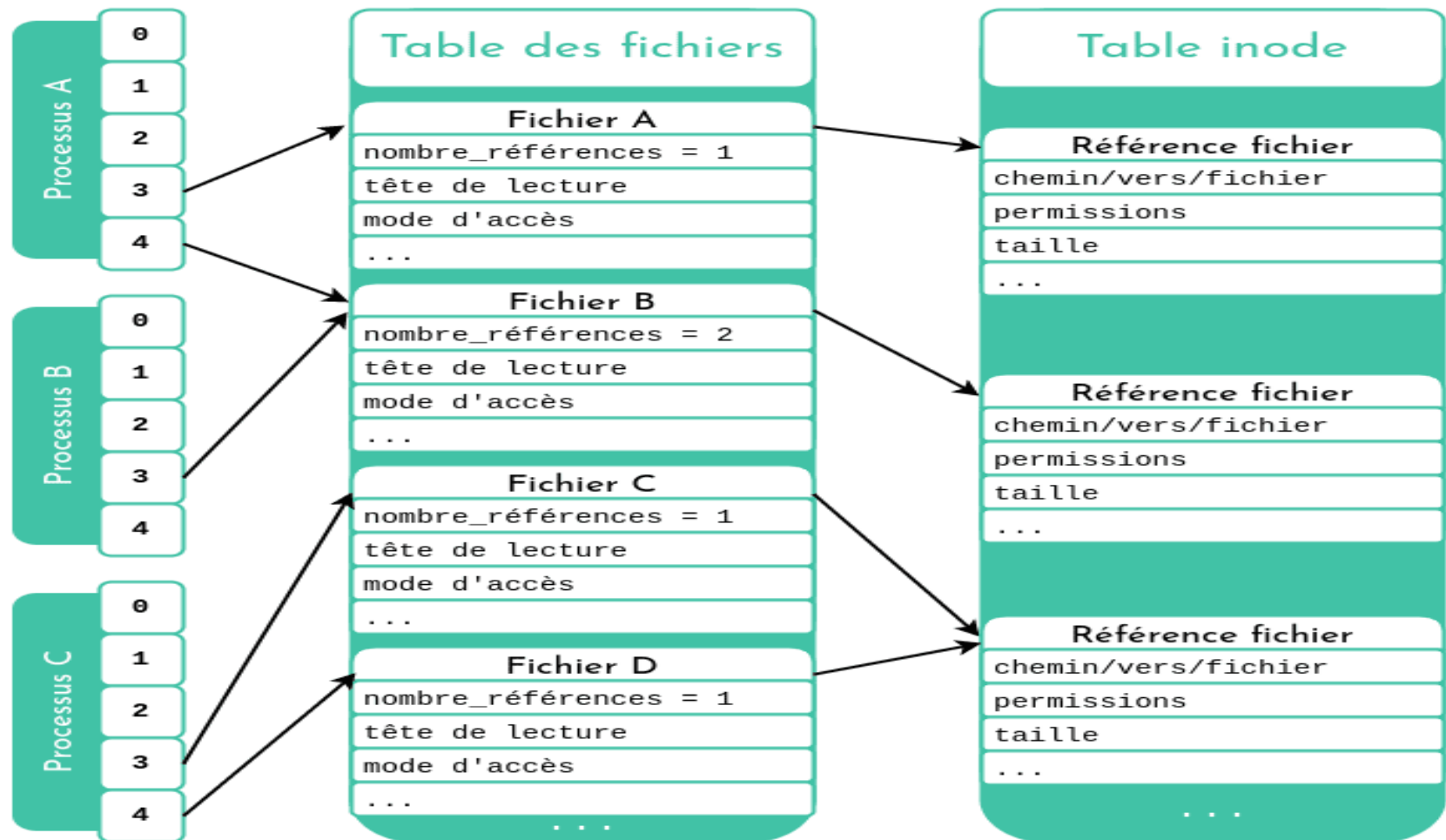
Les Descripteurs:

Descripteurs

- ▶ Avant de réaliser une E/S, un processus doit acquérir les droits en mode
 - ▶ lecture, ou
 - ▶ écriture, ou
 - ▶ lecture-écriture.
- ▶ Il le fait en demandant une **ouverture** du fichier, dans un des 3 modes.
- ▶ En cas de succès, le système fournit au programmeur un identifiant entier, appelé **descripteur** permettant les accès ultérieurs.

Descripteurs

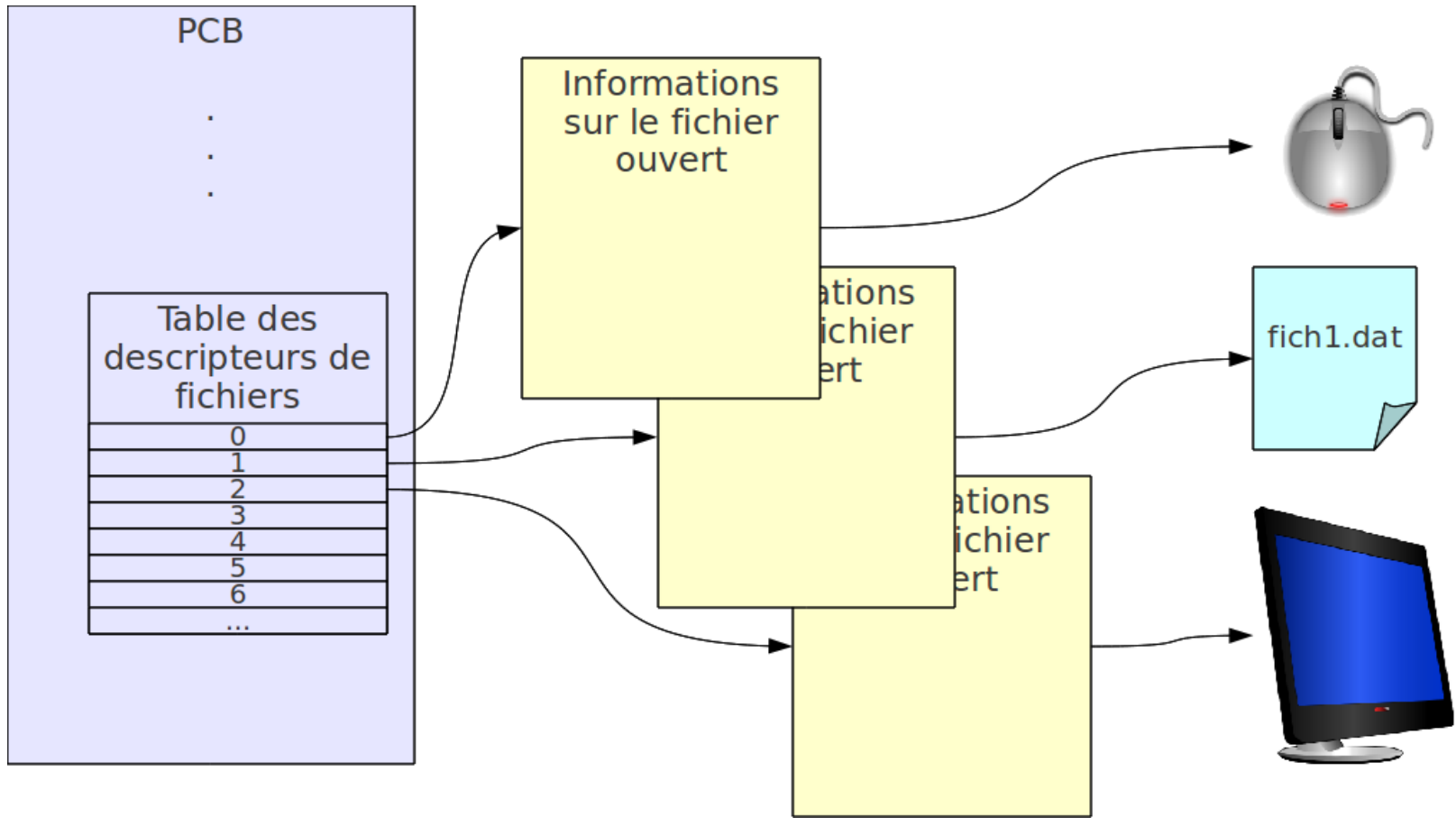
Un fichier est accessible à travers un descripteur local à un processus qui permet de le référencer indépendamment de son nom physique. Ce descripteur permet d'accéder à une table qui indique l'ensemble des fichiers ouverts. Plus exactement, chaque processus possède une table de descripteurs, un pour chaque fichier ouvert, selon le schéma:



Descripteurs

- ▶ Un descripteur obtenu par un processus lui est propre.
- ▶ Il donne au processus le droit d'accès à une entrée dans une « **table des fichiers ouverts** » (TFO).
- ▶ Un même fichier peut être ouvert plusieurs fois (dans des modes éventuellement différents) par un ou plusieurs processus.
- ▶ 3 descripteurs spéciaux :
 - ▶ **STDIN_FILENO**, entrée standard,
 - ▶ **STDOUT_FILENO**, sortie standard,
 - ▶ **STDERR_FILENO**, sortie erreur standard.

Descripteurs



Les tubes anonymes

- Un processus peut envoyer un message à un autre processus se trouvant sur la même machine ou sur des machines différentes.
- Il existe plusieurs mécanismes de communication par envoi de messages :
 - les tubes de communication,
 - les files de messages
 - les sockets....
- Les tubes de communication permettent à deux ou plusieurs processus s'exécutant sur une même machine d'échanger des informations.
- On distingue deux types de tubes :
 - Les tubes anonymes (unnamed pipe),
 - Les tubes nommés (named pipe) qui ont une existence dans le système de fichiers (un chemin d'accès).

Communication Inter-processus:

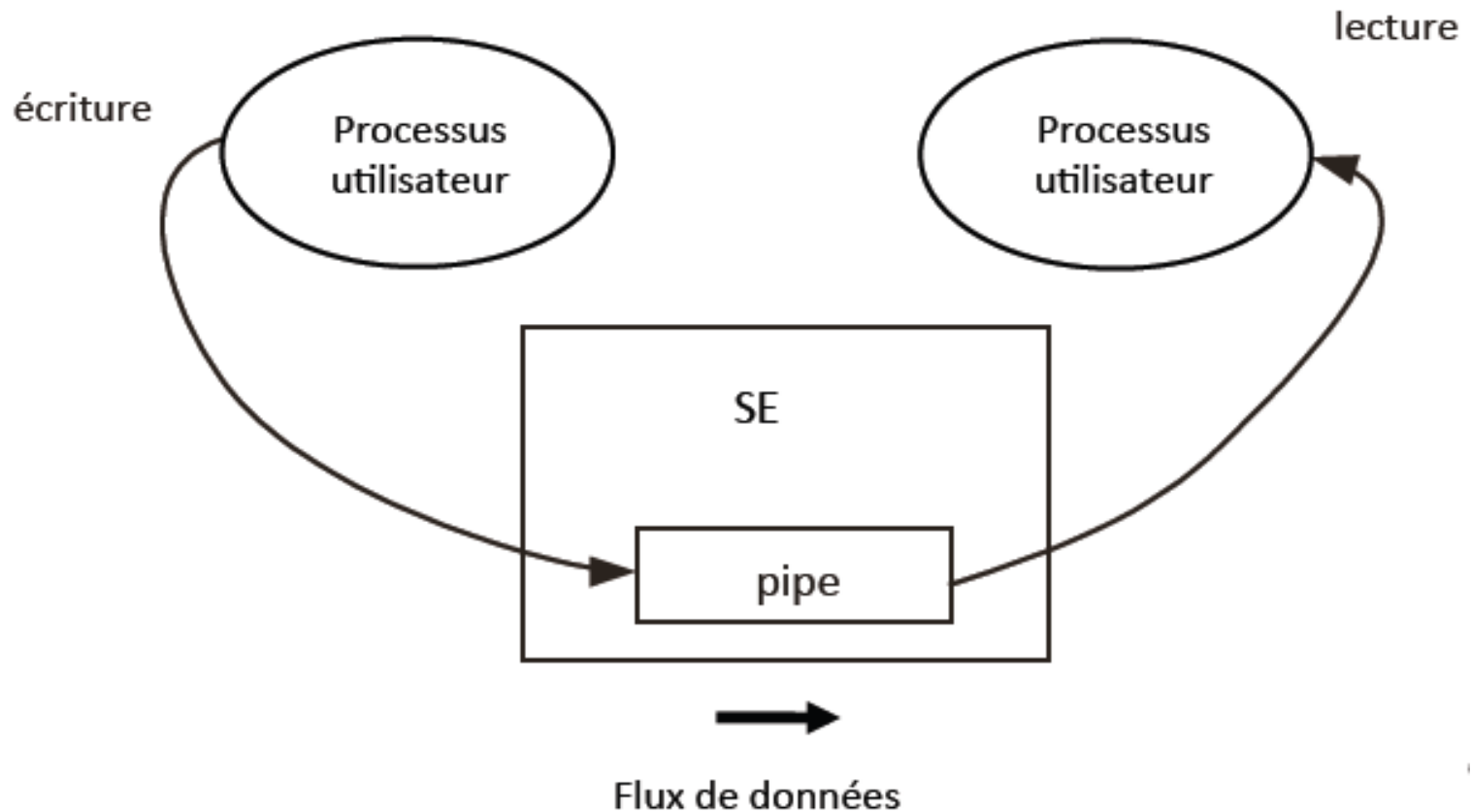
Les tubes anonymes

Les tubes anonymes

- Les tubes anonymes (pipes) peuvent être considérés comme des fichiers temporaires.
- Ils permettent d'établir des liaisons unidirectionnelles de communication entre processus dépendants.
- Un tube de communication permet de mémoriser des informations et se comporte comme une file FIFO.
- Il est caractérisé par deux descripteurs de fichiers (lecture et écriture) et sa taille limitée (PIPE_BUF) est approximativement égale à 4KO.
- L'opération de lecture dans un tube est destructrice : une information ne peut être lue qu'une seule fois d'un tube.
- Lorsque tous les descripteurs du tube sont fermés, le tube est détruit.
- Les tubes anonymes peuvent être créés par :
 - l'opérateur du shell « | »
 - l'appel système `pipe( )`.



Les tubes anonymes

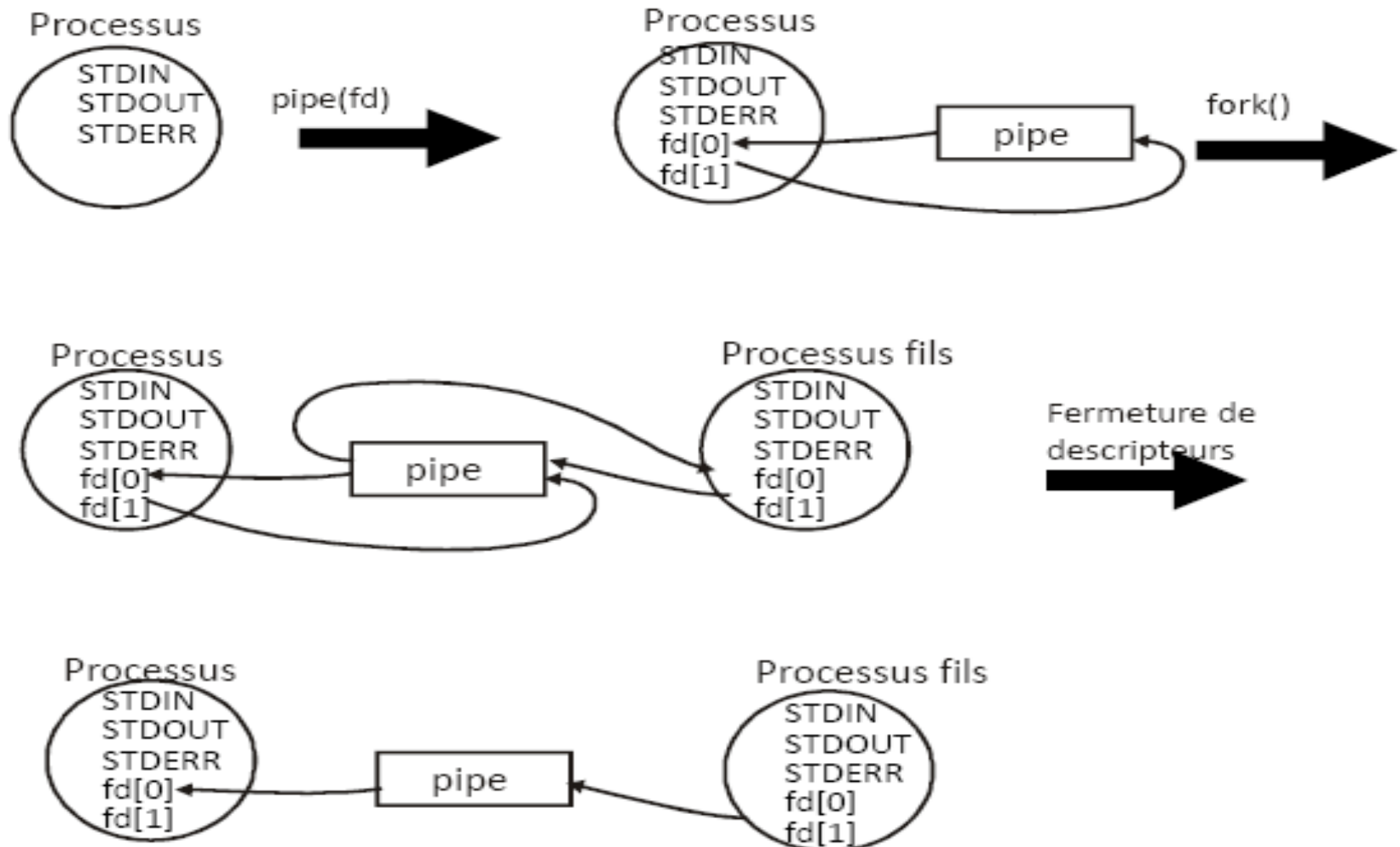


Les tubes anonymes

- Un tube de communication anonyme est créé par l'appel système :
`int pipe(int p[2]).`
- Cet appel système crée deux descripteurs de fichiers. Il retourne, dans `p`, les descripteurs de fichiers créés :
 - `p[0]` contient le descripteur réservé aux lectures à partir du tube
 - `p[1]` contient le descripteur réservé aux écritures dans le tube.
- Les descripteurs créés sont ajoutés à la table des descripteurs de fichiers du processus appelant.
- Seul le processus créateur du tube et ses descendants (ses fils) peuvent accéder au tube (duplication de la table des descripteurs de fichiers).
- Si le système ne peut pas créer de tube pour manque d'espace, l'appel système `pipe()` retourne la valeur `-1`, sinon il retourne la valeur `0`.
- L'accès au tube se fait via les descripteurs (comme pour les fichiers ordinaires).



Les tubes anonymes

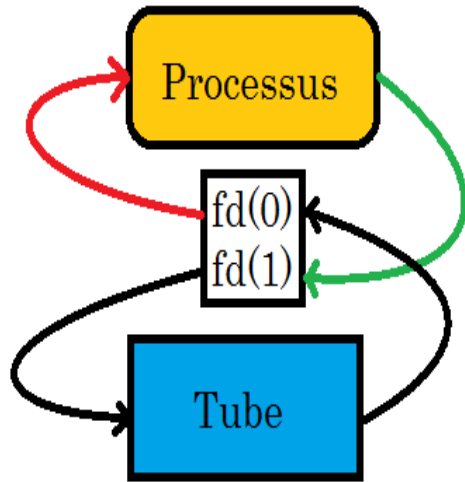


Les tubes anonymes

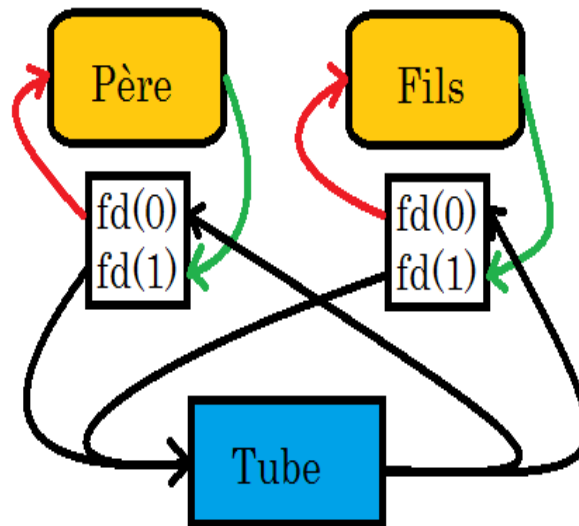
- Les tubes anonymes sont utilisés pour la communication entre un processus père et ses processus fils, avec un processus qui écrit sur le tube, appelé processus écrivain, et un autre qui lit à partir du tube, appelé processus lecteur.
- La séquence d'événements pour une telle communication est comme suit :
 1. Le processus père crée un tube de communication anonyme en utilisant l'appel système `pipe( )` ;
 2. Le processus père crée un ou plusieurs fils en utilisant l'appel système `fork( )` ;
 3. Le processus écrivain ferme le descripteur de fichier, non utilisé, de lecture du tube ;
 4. De même, le processus lecteur ferme le descripteur de fichier, non utilisé, d'écriture du tube ;
 5. Les processus communiquent en utilisant les appels système:
`read(fd[0], buffer, n)` et `write(fd[1], buffer, n)`;
 1. Chaque processus ferme son fichier lorsqu'il veut mettre fin à la communication via le tube.



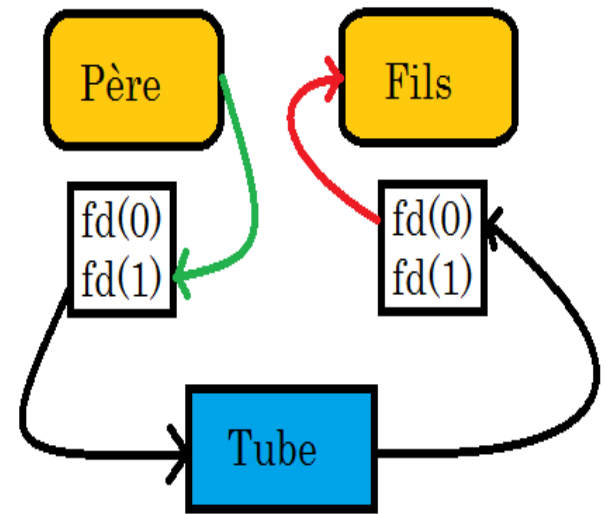
Les tubes anonymes



Après pipe(fd)



Après fork()
Les descripteurs sont copiés



Après fermeture des
descripteurs inutiles

Les tubes anonymes

1. **pipe(descripteur)** :

Crée un tube (un canal de communication unidirectionnel) et retourne 0 en cas de succès, ou -1 en cas d'échec.

```
int fd[2];  
pipe(fd);
```

Après un appel réussi :

- ❑ **fd[0]**: est utilisé pour la lecture depuis le tube.
- ❑ **fd[1]**: est utilisé pour l'écriture dans le tube.

2. **write(descripteur, buffer, longueur)** :

Écrit un certain nombre d'octets (défini par longueur) à partir de la zone mémoire pointée par buffer dans le fichier ou le tube désigné par le descripteur de fichier descripteur.

Dans le contexte des tubes, si le descripteur pour l'écriture est p[1] (où p est le tableau créé par l'appel à pipe), on peut écrire une valeur dans le tube en utilisant ce descripteur.

```
write(fd[1], &v, sizeof(float));
```

- La fonction retourne le nombre d'octets écrits ou -1 en cas d'erreur.

Les tubes anonymes

3. `read(descripteur, buffer, longueur):`

Lit jusqu'à longueur octets dans le tube associé au descripteur en lecture et place les octets dans la mémoire pointée par buffer.

```
read(fd[0], &v, sizeof(float));
```

- La fonction renvoie le nombre d'octets lus, ou 0 si le tube est vide et qu'il n'y a plus d'écrivains, ou encore -1 en cas d'erreur

4. `close(descripteur):`

ferme le descripteur de tube en lecture ou en écriture, selon celui passé en argument, pour le processus qui a appelé close.

```
close (fd[0]);
```

- La fonction renvoie 0 en cas de succès ou -1 en cas d'échec.

Exemple 1

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int tube[2];
char buf[20];
main() {
    pipe(tube);
    if (fork()==0) { /* fils */
        close(tube[0]);
        write(tube[1], "bonjour", 8);
    }
    else { /* pere */
        wait(NULL);
        close(tube[1]);
        read(tube[0], buf, 8);
        printf("%s bien reçu\n", buf);
    }
}
```

Exemple 2

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define R 0 // Descripteur de lecture
#define W 1 // Descripteur d'écriture

int main() {
    int pid;
    int fd[2]; // Tableau pour le pipe
    char message[100]; // Buffer pour le message reçu
    int Nboctets; // Nombre d'octets lus
    char* phrase = "VOICI MON MESSAGE POUR TOI PERE"; // Message fills

    system("clear"); // Nettoie le terminal
    printf("\t\tProcessus Courant: %d\n", (int)getpid());

    // Création du pipe
    if (pipe(fd) == -1) {
        perror("Création du pipe a échoué");
        exit(1);
    }

    pid = fork(); // Création du processus fils

    if (pid == -1) { // Vérification de l'échec du fork
        perror("fork a échoué");
        exit(2);
    }
}
```

Exemple 2-suite

```
if (pid == 0) { // Code exécuté par le fils
    close(fd[R]); // Fermeture de l'extrémité de lecture par le fils
    printf("Processus FILS: %d\n", (int)getpid());
    printf("FILS: PERE JE T'ENVOIE UN MESSAGE\n");

    // Envoi du message par le fils
    if (write(fd[W], phrase, strlen(phrase) + 1) == -1) {
        perror("write : Écriture dans le pipe a échoué");
        exit(4);
    }

    close(fd[W]); // Fermeture de l'extrémité d'écriture par le fils
    exit(3); // Terminaison du fils
}

// Code exécuté par le père
printf("J'attends la terminaison du fils\n");
wait(NULL); // Attente de la terminaison du fils

close(fd[W]); // Fermeture de l'extrémité d'écriture par le père

// Lecture du message dans le pipe
if ((Nboctets = read(fd[R], message, 100)) == -1) {
    perror("read : Lecture a échoué");
    exit(5);
}

message[Nboctets] = '\0'; // Ajout du caractère de fin de chaîne
printf("MESSAGE REÇU :: nbocets = %d, Message = %s\n", Nboctets, message);

close(fd[R]); // Fermeture de l'extrémité de lecture par le père

return 0;
}
```

Exemple 3

```
#include #include #include
int tube[2];
int m=12; int lu;
main()
{ pipe(tube);
  if (fork()==0) { /* fils */
    close(tube[1]);
    read(tube[0], &lu, sizeof(lu));
    printf("fils - valeur lue : %d\n", lu);
    close(tube[0]);
  }
  else { /* pere */
    close(tube[0]);
    m=m*3;
    write(tube[1], &m, sizeof(m));
    close(tube[1]);
    printf("pere- fin d'écriture\n");
    wait(NULL);
  }
}
```


Les tubes anonymes :

Redirection de stdin et stdout

Redirection de stdin et stdout

La duplication de descripteur, réalisée à l'aide des appels systèmes **dup** et **dup2**, permet à un processus de créer un nouveau descripteur de fichier qui est en fait un synonyme d'un descripteur déjà existant.

- **Dup():**

Créer et retourne un descripteur synonyme de **desc**. Le numéro associé au descripteur créé est le plus petit descripteur disponible dans la table des descripteurs de fichiers du processus.

```
#include <unistd.h>
int dup(int desc);
```

- **Dup2():**

Permet également de dupliquer un descripteur de fichier, mais avec plus de contrôle.

Dup2 transforme **desc2** en un descripteur synonyme a **desc1**

```
#include <unistd.h>
int dup2(int desc1,int desc2);
```

Exercice

Écrire un programme en langage C qui simule le comportement de la commande **ls -al | wc -l** en utilisant deux processus qui communiquent à travers un tube. Le processus parent exécutera **ls -al** et enverra sa sortie au processus enfant, qui exécutera **wc -l** pour compter le nombre total de lignes.

Pour réaliser cela, le processus parent devra rediriger sa sortie standard vers l'entrée du tube avant d'exécuter **ls -al**, tandis que le processus enfant devra rediriger son entrée standard depuis le tube avant d'exécuter **wc -l**. Utilisez la primitive **execlp** pour exécuter les commandes **ls** et **wc** respectivement.

Assurez-vous que votre programme produit le même résultat que **ls -al | wc -l**, c'est-à-dire le nombre total de fichiers et de répertoires répertoriés dans le répertoire courant.

Recouvrement d'un processus

La création d'un nouveau processus ne peut se faire que par le « recouvrement » du code d'un processus existant. Cela se fait à l'aide d'une des fonctions de la famille **exec** qui permet de faire exécuter par un processus un autre programme que celui d'origine. Lorsqu'un processus exécute un appel **exec**, il charge un autre programme exécutable en conservant le même environnement système :

- **Du point de vue du système**, il s'agit toujours du même processus : il a conservé le même pid.
- **Du point de vue de l'utilisateur**, le processus qui exécute **exec** disparaît, au profit d'un nouveau processus disposant du même environnement système et qui débute alors son exécution.

`int exec()`

permet de lancer et d'exécuter un programme externe en remplaçant le processus en cours. Différentes variantes pour la spécification du fichier et/ou des arguments existent : `execl`, `execle`, `execlp`, `execv`, `execvp`, ...

`execl (char *path, char *arg0, char *arg1, ..., char *argn, NULL)`

- cette variante exécute un programme ;
- `path` est le chemin du fichier programme ;
- `arg0` : premier argument (correspondant au nom du programme exécutable);
- `argn` est le `n` ième argument.

Exemple : `execl ("/bin/ls", "ls", "-la", NULL) ;`

- exécute la commande `ls`
- située dans le répertoire `/bin/`
- en utilisant le paramètre `-la`

Exemple : r1.c

```
#include<unistd.h>
int main() {
    printf("Je suis un programme qui va exécuter /bin/ls -l\n");
    execl("/bin/ls","ls","-l",NULL);
}
```

Recouvrement d'un processus : Résultats

```
Machine:~/Moi/Test> ls
d1  f1  f2
Machine:~/Moi/Test> ../r1
Je suis un programme qui va exécuter /bin/ls -l
total 4
drwxr-xr-x 2 blec sys 4096 2006-09-29 14:00 d1
-rw-r--r-- 1 blec sys    0 2006-09-29 14:00 f1
-rw-r--r-- 1 blec sys    0 2006-09-29 14:00 f2
Machine:~/Moi/Test>
```

Exercice

Donner l'organisation et écrire le programme C d'une application de transmission bidirectionnelle d'informations entre un processus père et un de ses fils via des tubes :

le père envoie 5 entiers au fils qui les affiche et renvoie ces entiers multipliés par 2. Ensuite Le père affiche ces doubles. correspondant.